

Fundamentos de programación en Python

Breve descripción:

El componente formativo aborda los fundamentos de la programación en Python, a partir del uso de variables, tipos de datos, estructuras compuestas, conversión de valores, funciones de entrada y salida, operadores, funciones integradas, módulos, librerías y estándares de codificación PEP 8. Estos elementos permiten comprender cómo se recibe, procesa y presenta la información en programas secuenciales básicos, de manera clara, ordenada y funcional.

Tabla de contenido

Introducción	4
1. Instalación y primeros pasos en Python	7
1.1. Proceso de instalación	9
1.2. Comentarios en Python	14
1.3. Errores de tecleo y excepciones	19
2. Variables: nombre, declaración y tipos de datos	21
2.1. Identificación del tipo de variable	28
2.2. Datos numéricos, booleanos y cadenas de caracteres (str).....	31
2.3. Operaciones aritméticas.....	44
3. Estructuras de datos compuestas y conversión de tipos	48
3.1. Listas, tuplas, rangos y diccionarios.....	49
3.2. Conjuntos	52
3.3. Conversión de tipos de datos	62
4. Entrada y salida de datos.....	68
4.1. Entradas estándar: concepto y función input()	69
4.2. Salidas estándar: función print() y técnicas de formato	76
5. Operadores y funciones integradas.....	85
5.1. Operadores aritméticos y precedencia.....	86

5.2. Operadores lógicos, relacionales y de cadena	91
5.3. Funciones integradas para entrada, salida y secuencias	97
6. Módulos, librerías y estándares de codificación.....	100
6.1. Concepto de módulo, librería y formas de importación.....	101
6.2. Estándares de codificación PEP 8	105
Síntesis	108
Glosario	109
Referencias bibliográficas	110
Créditos	111

Introducción

La construcción de programas secuenciales en Python requiere comprender los fundamentos que permiten recibir, almacenar, procesar y presentar información de manera clara. Estos elementos constituyen la base para escribir instrucciones organizadas, interpretar datos y desarrollar soluciones básicas orientadas a resolver necesidades concretas mediante código.

En la programación con Python, las variables cumplen un papel fundamental, ya que permiten guardar valores que pueden utilizarse durante la ejecución de un programa. A partir de ellas, es posible trabajar con diferentes tipos de datos, como números, cadenas de caracteres, valores booleanos y estructuras compuestas, los cuales facilitan la representación de información diversa dentro de una solución informática.

De igual manera, las funciones de entrada y salida permiten establecer la comunicación entre el programa y la persona que lo utiliza. La función `input()` facilita la captura de datos, mientras que `print()` permite mostrar resultados en pantalla. Estas operaciones se complementan con la conversión de tipos, los operadores, las funciones integradas, los módulos, las librerías y los estándares de codificación que orientan la escritura de programas claros y ordenados.

Para fortalecer la comprensión de los conceptos abordados y contextualizar su aplicación en la construcción de programas secuenciales en Python, se recomienda acceder al recurso audiovisual que acompaña este componente formativo.

Video 1. Fundamentos de programación en Python



[Enlace de reproducción del video](#)

Transcripción del video. Fundamentos de programación en Python

Iniciar en la programación con Python implica comprender cómo se construyen instrucciones claras para resolver problemas mediante datos, operaciones y resultados. Este lenguaje se caracteriza por una sintaxis sencilla, legible y flexible, lo que facilita el aprendizaje de conceptos básicos y su aplicación en programas secuenciales.

En este componente formativo se abordan los fundamentos necesarios para comenzar a programar en Python. Para ello, se parte de la instalación del entorno de trabajo, el uso de comentarios para documentar el código y la identificación de errores de tecleo y excepciones como parte del proceso de revisión y mejora.

Transcripción del video. Fundamentos de programación en Python

El recorrido continúa con el estudio de las variables, los tipos de datos numéricos, booleanos y cadenas de caracteres, así como las operaciones aritméticas. También se trabajan estructuras de datos compuestas, como listas, tuplas, rangos, diccionarios y conjuntos, útiles para almacenar, organizar y transformar información dentro de un programa.

Asimismo, se explican las funciones de entrada y salida, los operadores aritméticos, lógicos y relacionales, y las funciones integradas que permiten procesar datos de manera más eficiente. Finalmente, se presentan los módulos, las librerías y los estándares de codificación PEP 8 como buenas prácticas para escribir código claro, ordenado y fácil de mantener.

Así, los fundamentos de programación en Python se convierten en una base práctica para crear soluciones de software aplicadas al contexto colombiano. Su aprendizaje permite organizar datos, automatizar procesos sencillos y desarrollar programas secuenciales que respondan a necesidades reales de formación, trabajo, emprendimiento y transformación digital en diferentes sectores.

1. Instalación y primeros pasos en Python

El aprendizaje de Python inicia con la preparación del entorno de trabajo, ya que este permite escribir, ejecutar y verificar instrucciones de código de manera organizada. Contar con una instalación adecuada facilita el desarrollo de ejercicios prácticos, la revisión de errores y la comprensión progresiva de los fundamentos de programación.

Python es un lenguaje de programación de propósito general, reconocido por su sintaxis clara, su legibilidad y su amplia aplicación en áreas como desarrollo de software, análisis de datos, automatización e inteligencia artificial. Estas características lo convierten en una herramienta adecuada para iniciar la construcción de programas secuenciales, en los que las instrucciones se ejecutan paso a paso.

Antes de iniciar la instalación, es necesario descargar el instalador desde el sitio oficial de Python: <https://www.python.org/downloads/>.

Esta fuente garantiza que el archivo corresponda a una versión reciente, estable y libre de modificaciones no autorizadas. El uso de páginas no oficiales puede exponer el equipo a instaladores desactualizados o con código malicioso.

La preparación inicial en Python requiere aplicar buenas prácticas que facilitan la escritura, comprensión y corrección del código. Entre las más importantes se encuentran:

- **Usar comentarios:** permiten explicar partes importantes del código y facilitar su comprensión durante la revisión o modificación del programa.
- **Leer mensajes de error:** ayuda a identificar fallas de escritura, sintaxis o ejecución, y orienta la corrección del código.

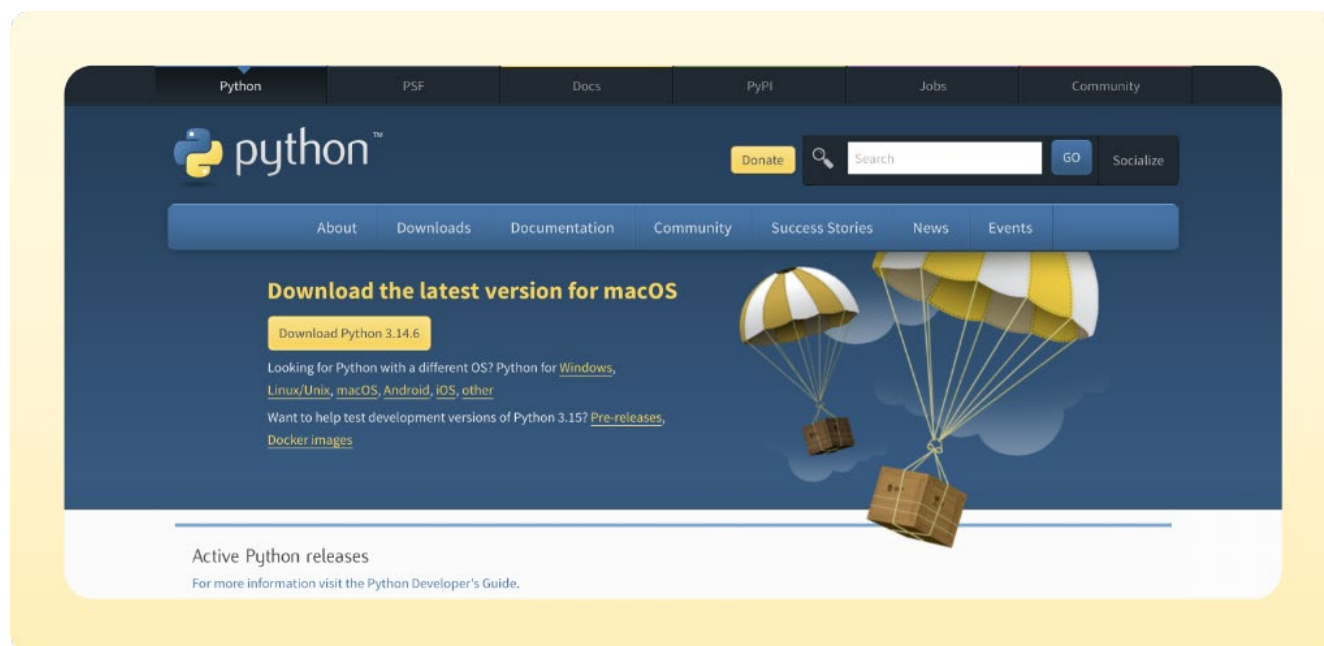
- **Escribir código claro:** favorece que las instrucciones sean comprensibles, ordenadas y fáciles de seguir durante la ejecución del programa.
- **Mantener buenas prácticas:** contribuye a crear soluciones más organizadas, legibles y sencillas de actualizar.

Estas prácticas fortalecen el proceso de aprendizaje, porque permiten comprender mejor el funcionamiento del código y corregir errores de manera progresiva antes de avanzar hacia la escritura de instrucciones básicas en Python.

Con esta preparación, se orienta la instalación de Python y la verificación del entorno de trabajo, de modo que las herramientas necesarias queden disponibles antes de escribir las primeras instrucciones.

Como apoyo visual, la Figura 1 muestra la página oficial de descargas de Python. Esta referencia permite reconocer el sitio correcto, aunque su apariencia puede variar con las actualizaciones de la plataforma.

Figura 1. Sitio oficial para la descarga de Python



Nota. Tomada de Python Software Foundation (s.f.-a).

1.1. Proceso de instalación

La instalación de Python permite preparar el entorno de trabajo necesario para escribir, ejecutar y comprobar instrucciones básicas de programación. En este proceso se incorpora el intérprete del lenguaje al equipo, herramienta que permite que las instrucciones escritas en Python puedan ser interpretadas y ejecutadas correctamente.

Antes de iniciar, es importante contar con conexión a internet, permisos de administrador y acceso al sitio oficial de Python. Estos elementos ayudan a realizar una instalación segura, estable y adecuada para comenzar el trabajo práctico con el lenguaje.

A continuación, se presentan los pasos para realizar la instalación en un equipo con sistema operativo Windows:

a) Paso 1. Acceder al sitio oficial de Python

Ingresa al sitio web oficial del lenguaje a través de la URL

<http://www.python.org>.

Es fundamental utilizar esta página oficial para garantizar que el instalador descargado sea seguro y corresponda a la versión más reciente y estable del lenguaje.

b) Paso 2. Descargar el instalador ejecutable

En la página oficial de descargas, ubicar la sección “Descargar la última versión para Windows”. Luego, seleccionar el enlace Python 3.14.6, disponible en la opción “O consiga el instalador independiente para Python 3.14.6”. Esta opción permite obtener el archivo ejecutable para instalar la versión indicada en un equipo con sistema operativo Windows.

Si el equipo utiliza otro sistema operativo, se debe seleccionar el enlace correspondiente en la línea “Python para Windows, Linux/Unix, macOS, Android, iOS u otros”, según la necesidad de instalación.

c) Paso 3. Iniciar la instalación

Una vez descargado el archivo, localizarlo en la carpeta de descargas y ejecutarlo haciendo doble clic sobre él. Se abrirá la ventana del asistente de instalación de Python.

d) Paso 4. Seleccionar la casilla "Add Python to PATH"

Antes de continuar con la instalación, es indispensable marcar la casilla "Add Python to PATH" que aparece en la parte inferior de la ventana inicial del instalador.

Esta opción es clave, ya que permite ejecutar Python desde cualquier ubicación en la línea de comandos sin necesidad de escribir la ruta completa del ejecutable. Omitir este paso obligará posteriormente a configurar las variables de entorno de forma manual.

e) Paso 5. Completar el proceso de instalación

Seleccionar la opción de instalación recomendada (Install Now) y esperar a que el asistente finalice el proceso. Al terminar, se mostrará un mensaje de confirmación que indica que la instalación fue exitosa.

f) Paso 6. Comprobar la instalación desde la línea de comandos

Para verificar que Python quedó correctamente instalado, realizar las siguientes acciones:

- Abrir la línea de comandos de Windows ejecutando el programa cmd.exe (puede buscarse desde el menú Inicio escribiendo "cmd").
- En la ventana de la consola, escribir la palabra Python y presionar la tecla ENTER.
- Si la instalación es correcta, se iniciará el entorno interactivo de Python, mostrando la versión instalada y el símbolo del sistema >>>, donde es posible introducir comandos y probar instrucciones del lenguaje.

g) Paso 7. Salir del entorno interactivo

Para cerrar la sesión del intérprete interactivo y regresar a la línea de comandos de Windows, utilizar el comando quit() y presionar ENTER.

Además del intérprete estándar, Python puede ejecutarse en entornos locales y en plataformas en la nube. La elección depende del tipo de práctica, la disponibilidad

de internet, el nivel de configuración requerido y la forma como se desea organizar el trabajo con el código.

A continuación, se describen las principales alternativas para ejecutar Python y sus usos recomendados según el tipo de actividad:

- **Entornos locales:** son herramientas instaladas en el equipo, como Visual Studio Code, PyCharm o IDLE. Permiten trabajar sin depender totalmente de internet, guardar proyectos, depurar código y configurar extensiones, según necesidad técnica.
- **Entornos en la nube:** son plataformas accesibles desde navegador, como Google Colab y Jupyter Notebook. Permiten ejecutar código Python, compartir archivos, trabajar colaborativamente y realizar prácticas sin instalar programas en el equipo local previamente.
- **Uso recomendado de herramientas locales:** convienen cuando se requiere administrar carpetas del proyecto, instalar extensiones, usar terminal integrada, depurar archivos completos y conservar configuraciones propias del equipo de trabajo local.
- **Uso recomendado de la nube:** conviene para practicar rápidamente, compartir cuadernos, ejecutar ejemplos guiados y trabajar con datos, sin depender de instalaciones previas ni configuraciones avanzadas del equipo personal local.

Comprender estas alternativas permite seleccionar el entorno más adecuado según el tipo de actividad, los recursos disponibles y el nivel de configuración requerido. En este contexto, Visual Studio Code se presenta como una opción local

recomendada para escribir, ejecutar y revisar programas en Python de manera organizada.

A. Instalación de Visual Studio Code

Visual Studio Code, también conocido como VS Code, es un editor de código gratuito y multiplataforma desarrollado por Microsoft. Su uso facilita la escritura, revisión y ejecución de programas en Python mediante herramientas como resaltado de sintaxis, terminal integrada, depuración y extensiones.

En el trabajo con Python, VS Code permite organizar archivos, ejecutar instrucciones y revisar errores desde un mismo entorno. Esta integración favorece una práctica más ordenada, ya que el código, la consola y los mensajes de verificación pueden consultarse en una sola interfaz.

Para utilizar Python en VS Code, es necesario instalar la extensión oficial del lenguaje. Esta extensión permite ejecutar archivos, reconocer la sintaxis, apoyar la depuración y aplicar buenas prácticas de codificación. Con ello, el editor se convierte en un entorno adecuado para iniciar la construcción de programas secuenciales.

Para reforzar la instalación y configuración del entorno de trabajo, se recomienda consultar el recurso audiovisual sobre el uso de Visual Studio Code con Python. Este material orienta, de manera visual, el procedimiento para preparar el editor y ejecutar los primeros programas. <https://www.youtube.com/watch?v=gEg1lxY5ioA>

El uso de un editor de código facilita la escritura organizada de instrucciones y la revisión de los resultados obtenidos. Esta base permite avanzar hacia una práctica esencial en programación, documentar el código mediante comentarios que expliquen su propósito sin afectar la ejecución del programa.

1.2. Comentarios en Python

Los comentarios permiten documentar el propósito de una instrucción sin afectar la ejecución del programa. Su uso resulta útil para explicar decisiones, registrar observaciones y facilitar que otras personas comprendan el código cuando deban revisarlo o modificarlo.

Un comentario es una línea de texto no ejecutable; esto significa que el intérprete de Python no la toma como una instrucción del programa. Por esta razón, los comentarios ayudan a mejorar la claridad del código sin cambiar los resultados de la ejecución.

En Visual Studio Code, los comentarios se escriben dentro del archivo de Python con extensión .py. Para ejecutarlos, se guarda el archivo y se usa la terminal integrada del editor. El intérprete omite los comentarios y solo ejecuta las instrucciones de código.

Hay dos formas de escribir comentarios en Python:

- a. **Comentarios de una línea:** se escriben con el símbolo # al inicio de la línea. Se utilizan para explicar una instrucción breve o una acción específica del programa.

esto es una línea de comentario

- b. **Comentarios de varias líneas:** pueden escribirse usando varias líneas iniciadas con # o mediante comillas triples al inicio y al final del bloque de texto, cuando se requiere una explicación más amplia.

Autor: Pepito Pérez

```
# Ciudad: Bogotá
```

```
>>> x = 0
```

```
>>> y = 8 # Los comentarios también pueden estar dentro de una línea
```

```
""" Este es un comentario multilínea.
```

```
Podemos escribir varias líneas. """
```

Para comprobar su funcionamiento en VS Code, se puede crear un archivo llamado `comentarios.py`, copiar el código anterior y ejecutarlo desde la terminal. Al hacerlo, Python ignorará los comentarios y procesará únicamente las instrucciones ejecutables.

Los comentarios fortalecen la comprensión y el mantenimiento del código. Para usarlos de manera adecuada, se recomienda tener en cuenta los siguientes aspectos:

- **Lectura del código:** facilitan que otras personas comprendan la lógica del programa sin revisar cada instrucción en detalle.
- **Revisión y corrección:** ayudan a identificar la intención de una línea o bloque de código durante procesos de ajuste, prueba o depuración.
- **Mantenimiento del programa:** permiten actualizar, ampliar o modificar el código con mayor facilidad, especialmente cuando el proyecto crece o es revisado tiempo después.
- **Claridad de las explicaciones:** deben redactarse como oraciones breves, completas y pertinentes, sin repetir de forma innecesaria lo que el código ya expresa.

- **Actualización permanente:** deben revisarse cuando el código cambia, para evitar explicaciones desactualizadas o inconsistentes con la lógica implementada.

El uso adecuado de comentarios contribuye a escribir programas más legibles, ordenados y fáciles de mantener. Por esta razón, deben emplearse como apoyo a la comprensión del código y no como una repetición literal de cada instrucción.

Para fortalecer la comprensión sobre el uso de comentarios en Python, se presenta el siguiente pódcast instruccional. Este recurso explica, de manera sencilla y aplicada, cómo los comentarios ayudan a documentar instrucciones, aclarar la intención del código y facilitar su revisión en un entorno de programación como Visual Studio Code.

Transcripción del pódcast. Comentarios que ayudan a entender el código

Mujer: en programación, un código fácil de entender vale tanto como un código que funciona correctamente. Con el paso del tiempo, entender cómo está construido facilita cualquier ajuste o corrección.

Hombre: los comentarios en Python ayudan a facilitar esa comprensión. Son pequeñas anotaciones que explican el propósito del código sin alterar su funcionamiento.

Mujer: aunque hacen parte del archivo, los comentarios no son instrucciones que Python deba ejecutar. Simplemente los ignora porque están dirigidos a las personas que leen el código.

Hombre: muchas veces esos comentarios terminan ayudando al mismo programador. Después de varios días o semanas resulta mucho más fácil recordar por qué una parte del código fue escrita de determinada manera.

Hombre: la forma más sencilla de crear un comentario es utilizando numeral o almohadilla. Todo lo que se escriba después será tomado como una explicación y no como una instrucción del programa.

Mujer: cuando la explicación necesita más espacio, también pueden utilizarse varios comentarios consecutivos o bloques entre comillas triples para describir archivos completos o secciones importantes del programa.

Hombre: pensemos en un programa desarrollado en Visual Studio Code. Un comentario puede indicar dónde se registran los datos, dónde se realizan los cálculos o qué parte del código presenta los resultados. Esa información facilita cualquier revisión posterior.

Mujer: los comentarios resultan muy útiles cuando un algoritmo es complejo o una decisión de programación necesita una explicación adicional. Unas pocas palabras pueden evitar muchas dudas más adelante.

Hombre: eso sí, comentar el código no significa explicar todo. Un buen comentario aporta información que realmente ayuda. Si una instrucción ya es suficientemente clara, repetir lo mismo con otras palabras no aporta ningún beneficio.

Mujer: también conviene revisar los comentarios cada vez que el programa cambia. Una explicación desactualizada puede generar más confusión que el propio código porque describe algo que ya dejó de existir.

Hombre: cuando varias personas participan en el mismo proyecto, los comentarios facilitan el trabajo en equipo y permiten comprender un programa sin tener que interpretar cada línea desde el principio.

Hombre: un buen comentario evita que el programador tenga que volver a descubrir por qué escribió el código de determinada manera.

Mujer: escribir comentarios requiere apenas unos segundos, pero puede ahorrar mucho tiempo cuando el programa necesite revisarse, actualizarse o ampliarse.

Hombre: los buenos comentarios permanecen mucho más tiempo que la memoria del programador. Por eso siguen siendo una de las herramientas más valiosas para mantener un código claro y fácil de comprender.

El podcast presentado permite reforzar la importancia de los comentarios como apoyo para comprender y mantener el código escrito en Python. Su uso adecuado favorece la lectura del programa, evita confusiones durante la revisión y facilita que otras personas comprendan la lógica implementada.

Además de documentar el código mediante comentarios, es necesario aprender a interpretar los mensajes que Python muestra cuando una instrucción no se ejecuta correctamente. Estos mensajes permiten identificar errores de escritura, sintaxis o

ejecución, y orientan la corrección del programa de manera progresiva. Por ello, el siguiente apartado aborda los errores de tecleo y las excepciones como elementos clave para revisar, ajustar y mejorar el código.

1.3. Errores de tecleo y excepciones

Los mensajes de error deben leerse como información de diagnóstico. En Python, cada excepción indica una posible causa del problema y ayuda a localizar la instrucción que requiere revisión, por lo que su interpretación hace parte del proceso de aprendizaje de la programación.

Si se digita incorrectamente una expresión, Python lo indicará con un mensaje de error. El mensaje de error proporciona información acerca del tipo de error cometido y del lugar en el que ha sido detectado.

Código. Ejemplo de error de sintaxis.

```
>>> 1 + 2)
```

```
SyntaxError: unmatched ')'
```

En este ejemplo se ha cerrado un paréntesis cuando no había otro abierto previamente. Python indica un error de sintaxis (SyntaxError) y señala con el carácter ^ el lugar del error.

En Python, los errores que se producen durante el análisis o la ejecución de una instrucción se conocen como excepciones. Estas permiten identificar la causa del problema y orientar la corrección del código de manera más precisa.

La siguiente tabla presenta algunas excepciones frecuentes en Python y su descripción general. Este recurso facilita la lectura de los mensajes de error y ayuda a reconocer situaciones comunes durante la escritura de programas básicos.

Tabla 1. Excepciones comunes en Python

Tipo de excepción	Descripción
SyntaxError	Error en la estructura del código (paréntesis, comillas, indentación).
NameError	Se usa una variable que no ha sido definida previamente.
TypeError	Se aplica una operación a un tipo de dato incompatible.
ValueError	El valor pasado a una función no es válido, aunque el tipo sea correcto.
ZeroDivisionError	Se intenta dividir un número entre cero.
IndentationError	La indentación del código no es correcta.

Reconocer estas excepciones permite interpretar los mensajes de error con mayor seguridad y avanzar en la depuración del código. Por ello, se recomienda revisar la última línea del mensaje, donde Python suele indicar el tipo de excepción y una breve descripción del problema.

La revisión de errores y excepciones permite comprender que muchos problemas en Python se originan por el uso incorrecto de nombres, valores o tipos de datos. Por esta razón, después de reconocer cómo interpretar los mensajes del intérprete, es necesario estudiar las variables como elementos básicos para almacenar información, asignar valores y organizar los datos que serán procesados en un programa.

2. Variables: nombre, declaración y tipos de datos

Las variables son la base para almacenar y recuperar información durante la ejecución de un programa. En Python, su uso se apoya en nombres descriptivos, asignación de valores y reconocimiento del tipo de dato que contiene cada variable.

Comprender esta relación evita errores frecuentes al operar con números, textos o valores lógicos. Por esta razón, el tema inicia con las reglas de nombrado y continúa con la identificación de tipos, la clasificación de datos básicos y las operaciones aritméticas que permiten procesarlos.

En Python, las variables permiten almacenar y recuperar información durante la ejecución de un programa. Para comprender su uso, es necesario reconocer cómo se nombran, cómo reciben valores y cómo Python identifica automáticamente el tipo de dato asignado:

- **Variable:** es un nombre asociado a un valor almacenado en memoria durante la ejecución. Permite recuperar y modificar información según la lógica del programa. Por ejemplo, `nombre = "María"` guarda una cadena para usarla después en mensajes o procesos.
- **Asignación de valor:** corresponde al uso del signo igual (=) para asociar un dato con una variable. Por ejemplo, `edad = 25` indica que la variable `edad` almacena un número entero que puede utilizarse en operaciones posteriores.
- **Tipado dinámico:** significa que Python identifica el tipo de dato según el valor asignado, sin declararlo previamente. Por ejemplo, `precio = 2500.5` se reconoce como decimal, mientras que `activo = True` se interpreta como valor booleano.

- **Cambio de valor:** una variable puede recibir un nuevo dato durante la ejecución del programa. Por ejemplo, `estado = "pendiente"` y luego `estado = "finalizado"` muestran cómo el contenido puede actualizarse según el proceso realizado.

En conjunto, estos elementos permiten comprender cómo una variable almacena información y cómo Python interpreta el tipo de dato asignado. Su uso adecuado favorece la escritura de instrucciones claras, organizadas y fáciles de revisar durante la construcción de programas básicos.

Para evitar errores de sintaxis y facilitar la lectura del código, los nombres de variables deben cumplir reglas básicas de escritura. Estas reglas permiten construir identificadores claros, válidos y coherentes con las convenciones del lenguaje Python:

- Debe comenzar con una letra (a-z, A-Z) o con el carácter guion bajo (`_`).
- Puede contener letras, dígitos (0-9) y guiones bajos.
- No puede comenzar con un dígito.
- No puede ser una palabra reservada del lenguaje (`if`, `else`, `for`, `while`, `class`, `def`, `return`, `import`, `True`, `False`, `None`, entre otras).

Aplicar estas reglas desde el inicio favorece la escritura de código legible y evita errores de sintaxis asociados a nombres mal contruidos o palabras reservadas.

Las palabras reservadas son términos propios del lenguaje Python que tienen una función específica dentro de su sintaxis. Por esta razón, no deben utilizarse como nombres de variables, funciones, clases u otros identificadores. En la versión instalada de Python, la lista activa puede consultarse directamente desde el intérprete mediante el módulo `keyword`.

Código. Palabras reservadas de Python

```
import keyword  
  
print(keyword.kwlist)
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class',  
'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in',  
'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

Este comando muestra las palabras reservadas reconocidas por la versión de Python instalada en el equipo. Consultar esta lista permite verificar qué términos no deben usarse como nombres de variables y ayuda a evitar errores de sintaxis durante la escritura del código.

Para reconocer con mayor claridad qué nombres de variables acepta Python y cuáles generan error, es necesario revisar algunos criterios básicos de nomenclatura. Estos criterios ayudan a escribir identificadores válidos, evitar errores de sintaxis y mantener un código comprensible desde las primeras prácticas de programación:

- a) Diferenciar mayúsculas y minúsculas:** Python distingue entre edad, Edad y EDAD; por tanto, cada escritura representa una variable diferente.
- b) Usar nombres descriptivos:** el nombre de la variable debe indicar el dato que almacena. Por ejemplo, edad_usuario es más claro que x.
- c) Evitar nombres inválidos:** no se deben usar identificadores que inicien con números, contengan espacios, incluyan guion medio o coincidan con palabras reservadas del lenguaje.

d) Comparar ejemplos concretos: revisar nombres válidos e inválidos permite comprender las reglas de escritura y reconocer la causa de los errores más frecuentes al declarar variables.

Aplicar estos criterios favorece la escritura de código claro, ordenado y fácil de revisar. La Tabla 2 presenta ejemplos contrastados de nombres válidos e inválidos para fortalecer la comprensión de las reglas de nomenclatura en Python.

Tabla 2. Ejemplos de nombres válidos e inválidos de variables

Nombre válido	Nombre inválido
nombre	2nombre (comienza con dígito)
edad_usuario	edad-usuario (guion medio)
_total	class (palabra reservada)
valor1	mi variable (espacio)
CONSTANTE_PI	import (palabra reservada)

Estos ejemplos comparativos ayudan a escribir nombres de variables claros, correctos y compatibles con el intérprete, evitando errores desde la primera asignación.

En síntesis, un nombre de variable válido en Python solo puede contener letras, números y guion bajo, no debe comenzar con un dígito, ni coincidir con una palabra reservada. Respetar estas reglas evita errores de sintaxis y da lugar a un código más claro y fácil de mantener.

Usar nombres descriptivos desde las primeras prácticas permite reconocer con facilidad qué dato almacena cada variable. Esta decisión mejora la lectura del código, reduce errores de interpretación y facilita su revisión o mantenimiento.

Para aplicar las reglas de nomenclatura, es útil observar cómo se declaran variables con distintos tipos de datos. El siguiente ejemplo muestra nombres descriptivos asociados a cadenas de caracteres, números enteros, valores decimales y valores booleanos.

Código. Ejemplos de declaración y asignación de variables.

```
>>> nombre = "María"
```

```
>>> edad = 25
```

```
>>> _total = 100.5
```

```
>>> es_activo = True
```

```
>>> PI = 3.14159 # Convención: mayúsculas para constantes
```

En estos ejemplos, cada variable recibe un valor mediante el operador de asignación `=`. Además, se observa que Python identifica automáticamente el tipo de dato según el valor asignado: texto, número entero, número decimal o valor lógico.

Según el estándar PEP 8, se recomienda escribir los nombres de variables en `snake_case`, como `total_ventas` o `nombre_completo`, y reservar las mayúsculas sostenidas para constantes, como `MAX_INTENTOS`, `IVA` o `PI`.

Las variables en Python facilitan el almacenamiento, la modificación y el uso de datos durante la ejecución de un programa. Comprender sus características permite escribir instrucciones claras, evitar errores frecuentes y reconocer cómo el lenguaje interpreta la información asignada.

Para presentar estas propiedades de forma sintética, la siguiente figura organiza las características que ayudan a comprender el uso de las variables en Python dentro de un programa secuencial.

Figura 2. Características para comprender el uso de variables en Python



En conjunto, estas características muestran que una variable en Python no es un simple contenedor fijo, sino un nombre que puede recibir, cambiar y compartir información durante la ejecución del programa. Reconocerlas permite anticipar cómo se comportará una variable en distintas situaciones y escribir código con mayor seguridad.

Para comprender cómo se comportan las variables durante la ejecución de un programa, es necesario observar ejemplos prácticos que muestren la asignación, la reasignación y el intercambio de valores. Los siguientes códigos permiten reconocer cómo Python identifica automáticamente el tipo de dato, actualiza el contenido de una variable y facilita la asignación de varios valores en una sola instrucción.

Código. Asignación múltiple e intercambio de valores en Python

```
# Asignación múltiple
```

```
>>> a, b, c = 1, 2, 3
```

```
>>> print(a, b, c)
```

```
1 2 3
```

```
# Intercambio de valores
```

```
>>> a, b = b, a
```

```
>>> print(a, b)
```

```
2 1
```

Una vez comprendido que las variables pueden recibir, cambiar e intercambiar valores durante la ejecución del programa, es necesario reconocer qué tipo de dato

almacena cada una. Esta identificación permite validar si una variable contiene un número, una cadena de caracteres, un valor lógico u otro tipo de objeto, lo cual facilita la escritura de operaciones correctas y evita errores durante el procesamiento de la información.

2.1. Identificación del tipo de variable

Identificar el tipo de dato de una variable permite comprobar qué clase de información almacena durante la ejecución del programa. Esta verificación es importante porque Python interpreta de manera diferente un número entero, un valor decimal, una cadena de caracteres o un valor lógico.

En Python, las funciones `type()` e `isinstance()` permiten revisar el tipo de dato asociado a una variable. La función `type()` devuelve el tipo del objeto consultado, mientras que `isinstance()` permite comprobar si un valor pertenece a un tipo determinado, considerando también relaciones de herencia entre clases.

Para comprender su utilidad, es necesario diferenciar el propósito de cada función y reconocer en qué situaciones conviene aplicarlas:

- **Función `type()`:** permite conocer el tipo exacto de un objeto. Por ejemplo, si se declara `edad = 25`, la instrucción `type(edad)` devuelve `<class 'int'>`, porque el valor almacenado corresponde a un número entero que puede usarse en operaciones aritméticas.
- **Función `isinstance()`:** permite verificar si un valor pertenece a un tipo de dato específico. Por ejemplo, `isinstance(edad, int)` devuelve `True` cuando

edad contiene un número entero, lo cual resulta útil antes de realizar cálculos o validar datos ingresados.

- **Validación de datos:** estas funciones ayudan a revisar si una variable contiene el tipo esperado antes de procesarla. Por ejemplo, antes de sumar una cantidad y un precio, puede comprobarse que ambos valores sean numéricos para evitar errores durante la operación.
- **Uso de `dir()`:** permite explorar nombres de atributos y métodos asociados a un objeto. Por ejemplo, `dir(str)` muestra operaciones disponibles para cadenas, como `lower`, `replace` o `split`; sin embargo, su resultado es una guía de exploración, no una lista estrictamente completa.

El uso de `type()`, `isinstance()` y `dir()` fortalece la revisión del código, porque permite reconocer la naturaleza de los datos y explorar operaciones disponibles. Esta práctica ayuda a prevenir errores y facilita la escritura de programas más claros y confiables.

Código. Uso de `type()` e `isinstance()` para identificar tipos de datos.

```
edad = 25

precio = 3.14

mensaje = "Hola mundo"

activo = True

print(type(edad))

print(type(precio))

print(type(mensaje))
```

```
print(type(activo))  
  
print(isinstance(edad, int))  
  
print(isinstance(precio, float))  
  
print(isinstance(mensaje, str))  
  
print(isinstance(activo, bool))
```

Resultado esperado

```
<class 'int'>  
  
<class 'float'>  
  
<class 'str'>  
  
<class 'bool'>  
  
True  
  
True  
  
True  
  
True
```

Este ejemplo muestra que Python reconoce el tipo de dato según el valor asignado a cada variable. Además, permite comprobar si una variable pertenece a un tipo específico, lo cual resulta útil cuando se requiere validar información antes de realizar cálculos, comparaciones o conversiones.

Código. Exploración de métodos con `dir()`

```
print(dir(str))
```

Resultado esperado parcial

```
['capitalize', 'casefold', 'center', 'count', 'encode',  
'endswith', 'find', 'format', 'index', 'isalnum',  
'isalpha', 'lower', 'replace', 'split', 'strip', 'upper']
```

La función `dir()` permite explorar nombres asociados a un objeto, como métodos y atributos. En este caso, al consultar `dir(str)`, se identifican operaciones disponibles para cadenas de caracteres. Esta exploración sirve como apoyo para reconocer qué acciones pueden aplicarse a cada tipo de dato.

Después de reconocer cómo se identifica el tipo de dato de una variable, es necesario estudiar los datos básicos más utilizados en Python. Esta base permite comprender cómo funcionan los números, los valores booleanos y las cadenas de caracteres, así como las operaciones que pueden realizarse con cada uno de ellos.

2.2. Datos numéricos, booleanos y cadenas de caracteres (str)

Los tipos de datos en Python permiten representar la información que utiliza un programa durante su ejecución. Cada tipo cumple una función específica y orienta las operaciones que pueden realizarse con los valores almacenados, como calcular, comparar, validar, recorrer o mostrar información.

Para comprender su uso inicial, es útil reconocer las categorías más frecuentes mediante una definición breve y un ejemplo de asignación. Esta revisión facilita la selección del tipo de dato adecuado según la naturaleza de la información que se necesita procesar:

- **Datos numéricos:** representan cantidades usadas en cálculos. Incluyen enteros (int), decimales (float) y complejos (complex). Por ejemplo, `edad = 25`, `precio = 1250.50` y `z = 5 + 3j` almacenan valores numéricos con diferentes niveles de precisión.
- **Datos booleanos:** representan valores de verdad. Solo admiten True o False, por lo que son útiles para validar condiciones. Por ejemplo, `activo = True` puede indicar que un usuario está habilitado, mientras que `pago_realizado = False` señala una acción pendiente.
- **Cadenas de caracteres:** representan información textual mediante el tipo str. Se emplean para nombres, mensajes, códigos o descripciones. Por ejemplo, `nombre = "María"` almacena texto que puede mostrarse, compararse, transformarse o combinarse con otros valores dentro del programa.
- **Secuencias:** agrupan varios elementos en un orden determinado. Incluyen listas (list), tuplas (tuple) y rangos (range). Por ejemplo, `notas = [4.0, 3.8, 5.0]` permite almacenar varias calificaciones para consultarlas o recorrerlas durante la ejecución.
- **Conjuntos:** almacenan elementos únicos mediante el tipo set. Son útiles para eliminar duplicados y comparar grupos de datos. Por ejemplo,

ciudades = {"Cali", "Bogotá", "Cali"} conserva cada ciudad una sola vez, aunque se haya escrito repetida.

- **Mapas o diccionarios:** almacenan información en pares clave-valor mediante el tipo dict. Facilitan asociar un dato con su significado. Por ejemplo, persona = {"nombre": "Ana", "edad": 30} permite consultar la edad o el nombre usando sus claves.
- **Valor nulo:** representa ausencia de valor mediante None. Se utiliza cuando una variable aún no tiene información asignada. Por ejemplo, resultado = None puede indicar que el dato será calculado o recibido en una etapa posterior del programa.

Reconocer estas categorías permite comprender cómo Python organiza la información antes de procesarla. Esta base favorece la escritura de instrucciones coherentes, evita errores de tipo y facilita la selección de estructuras adecuadas para cada necesidad del programa.

La siguiente tabla reúne los tipos de datos más utilizados en Python, su palabra clave y un ejemplo sencillo de asignación. Esta síntesis funciona como una referencia rápida para comparar las categorías y reconocer cómo se declara cada valor dentro del lenguaje.

Tabla 3. Resumen de los tipos de datos en Python

Tipo de dato	Palabra clave	Uso principal	Ejemplo
Entero	int	Representa números sin parte decimal.	edad = 25
Flotante	float	Representa números con parte decimal.	precio = 1250.50

Tipo de dato	Palabra clave	Uso principal	Ejemplo
Complejo	complex	Representa números con parte real e imaginaria.	$z = 5 + 3j$
Booleano	bool	Representa valores de verdad.	activo = True
Cadena de caracteres	str	Representa texto.	nombre = "María"
Valor nulo	NoneType	Representa ausencia de valor.	dato = None

Esta clasificación facilita la selección del tipo de dato adecuado según la información que se necesita almacenar, calcular, comparar o mostrar. Además, permite evitar errores de tipo al momento de combinar valores dentro de una operación.

A. Datos numéricos

Los datos numéricos permiten representar cantidades y realizar operaciones matemáticas. En Python, se utilizan principalmente los enteros, los flotantes y los complejos. Los enteros tienen precisión ilimitada según la memoria disponible; los flotantes representan números reales con precisión finita, y los complejos incluyen una parte real y una parte imaginaria.

- **Enteros (int):** representan números sin parte decimal. Se utilizan para contar elementos, edades, cantidades o posiciones. Por ejemplo, cantidad = 8 almacena un número entero que puede emplearse en sumas, restas, multiplicaciones o divisiones dentro de un programa.

- **Flotantes (float):** representan números con parte decimal. Son útiles para precios, promedios, temperaturas, porcentajes o medidas. Por ejemplo, `precio = 1250.50` almacena un valor decimal que puede utilizarse en cálculos donde se requiere mayor detalle numérico.
- **Complejos (complex):** representan números formados por una parte real y una parte imaginaria. Se utilizan en contextos matemáticos o científicos específicos. Por ejemplo, `z = 5 + 3j` permite acceder a la parte real con `z.real` y a la imaginaria con `z.imag`.

Los datos numéricos permiten representar cantidades con distintos niveles de complejidad como los valores enteros, decimales o complejos. Reconocer estas diferencias facilita la selección del tipo adecuado para realizar cálculos, expresar medidas, representar resultados matemáticos y evitar errores al operar con valores que tienen naturaleza distinta.

Las operaciones con números enteros permiten observar cómo Python calcula valores, asigna resultados a variables y muestra información en pantalla. En el siguiente código se realiza una suma a partir de una variable inicial y luego se aplica la función `round()` para redondear el resultado de una división.

Código. Operaciones con números enteros

```
>>> v = -3
```

```
>>> m = v + 8
```

```
>>> print(m)
```

5

```
>>> z = round(m/2)
```

```
>>> print(z)
```

```
2
```

Además de la forma decimal habitual, Python permite escribir números enteros en bases numéricas como binaria, octal y hexadecimal. Estas representaciones son útiles para comprender cómo un mismo valor puede expresarse de diferentes maneras en contextos relacionados con sistemas, memoria o procesamiento interno.

Código. Representación en binario, octal y hexadecimal

```
>>> decimal = 8
```

```
>>> binario = 0b1101 # 13
```

```
>>> octal = 0o11 # 9
```

```
>>> hexadecimal = 0xc # 12
```

Python también ofrece funciones integradas para convertir valores entre bases numéricas. Estas funciones permiten transformar un número decimal a binario, octal o hexadecimal, así como convertir una cadena escrita en una base específica a su equivalente decimal.

Código. Conversión entre bases numéricas

```
>>> bin(10) # '0b1010'
```

```
>>> oct(10) # '0o12'
```

```
>>> hex(10) # '0xa'
```

```
>>> int('1010', 2) # 10 (binario a decimal)
```

En Python, los números enteros son inmutables. Esto significa que cuando realizas una operación sobre una variable entera, no estás modificando el objeto original, sino creando uno nuevo con el resultado de la operación.

Los números de punto flotante permiten representar valores decimales. Sin embargo, su precisión es limitada debido a la forma como se almacenan internamente en el computador. Por esta razón, algunas operaciones pueden mostrar resultados con más cifras decimales de las esperadas.

Código. Precisión de los números flotantes

```
>>> real = 1.1 + 2.2
```

```
>>> print(real)
```

```
3.3000000000000003
```

```
>>> print(round(real, 1))
```

```
3.3
```

El resultado anterior muestra que los números flotantes pueden presentar pequeñas variaciones de precisión. Cuando se requiere exactitud en cálculos financieros o monetarios, se recomienda utilizar el módulo decimal, ya que permite controlar mejor la representación de valores decimales.

La notación científica permite escribir números muy grandes o pequeños de manera compacta. En Python, esta forma de representación utiliza la letra *e* para indicar potencias de diez, lo cual resulta útil en cálculos científicos, físicos o estadísticos.

Código. Notación científica en Python

```
# Notación científica
```

```
>>> avogadro = 6.022e23
```

```
>>> micro = 1.5e-6
```

Los números complejos se representan en Python con el tipo `complex`. Estos valores incluyen una parte real y una parte imaginaria, identificada con la letra `j`. El siguiente código permite consultar cada parte del número y calcular su módulo mediante la función `abs()`.

Código. Operaciones con números complejos

```
>>> complejo = 5+3j
```

```
>>> complejo.real # 5.0
```

```
>>> complejo.imag # 3.0
```

```
>>> abs(complejo) # 5.83 (módulo)
```

A diferencia de otros lenguajes, donde un entero puede estar limitado a 32 o 64 bits, en Python los enteros tienen precisión arbitraria. En la práctica, esto significa que pueden crecer según la memoria disponible en el equipo.

B. Datos booleanos

Los datos booleanos permiten representar condiciones dentro de un programa. En Python se expresan con el tipo `bool` y solo tienen dos valores posibles: `True`, cuando una condición se cumple, y `False`, cuando no se cumple.

- **Valor verdadero (True):** se usa cuando una condición se cumple. Por ejemplo, `usuario_activo = True` puede indicar que una cuenta está habilitada para ingresar al sistema.
- **Valor falso (False):** se usa cuando una condición no se cumple. Por ejemplo, `pago_realizado = False` puede indicar que una transacción aún no ha sido confirmada.
- **Valores equivalentes a falso:** Python interpreta como falsos valores como `None`, `False`, cero, cadenas vacías y colecciones vacías. Esta regla resulta útil al validar entradas o comprobar si una variable contiene información.

Estos valores permiten tomar decisiones dentro del programa, especialmente cuando se trabajan condiciones, validaciones o estructuras de control. Reconocer qué elementos se interpretan como verdaderos o falsos ayuda a evitar errores lógicos en la ejecución.

La declaración de valores booleanos permite comprobar cómo Python reconoce los tipos `bool` y `NoneType`. En el siguiente código se asignan valores lógicos y se verifica su tipo mediante la función `type()`.

Código. Declaración de variables booleanas y None

```
>>> a = False
```

```
>>> b = True
```

```
>>> type(a)
```

```
<class 'bool'>
```

```
>>> c = None
```

```
>>> type(c)
```

```
<class 'NoneType'>
```

Código. Conversión a booleano con bool()

```
>>> bool(0)    # False
```

```
>>> bool(1)    # True
```

```
>>> bool("")    # False
```

```
>>> bool("Hola") # True
```

```
>>> bool([])    # False
```

```
>>> bool([1, 2]) # True
```

El código muestra que los valores vacíos o nulos devuelven False, mientras que los valores con contenido devuelven True. Esta regla es fundamental al validar formularios, entradas del usuario o estructuras de datos antes de procesarlas.

C. Cadenas de caracteres

Las cadenas de caracteres permiten representar texto en Python. Se identifican con el tipo str y pueden contener letras, números, signos, espacios o símbolos. Se escriben entre comillas simples, dobles o triples, según la necesidad del contenido.

- **Texto entre comillas:** una cadena puede escribirse con comillas simples o dobles. Por ejemplo, nombre = "María" almacena texto y permite mostrarlo, compararlo o combinarlo con otros valores.
- **Comillas dentro del texto:** cuando una cadena contiene comillas internas, se pueden combinar comillas simples y dobles o usar barra invertida (\)

para escaparlas. Esto permite que Python las interprete como parte del texto.

- **Cadenas inmutables:** las cadenas no se modifican directamente. Cuando se transforma un texto, Python crea una nueva cadena con el resultado. Esta característica permite conservar el valor original y generar nuevas versiones del contenido.

Estas características permiten trabajar con datos textuales de manera clara y segura. Su comprensión facilita la construcción de mensajes, la validación de entradas y la transformación de información escrita dentro de un programa.

El siguiente código muestra diferentes formas de incluir comillas dentro de una cadena. Esta práctica permite escribir textos que contienen expresiones entre comillas sin generar errores de sintaxis.

Código. Escritura de cadenas con comillas

```
>>> saludo1 = 'Hola "María"'
>>> saludo2 = 'Hola \'María\''
>>> saludo3 = "Hola 'María'"
>>> print(saludo1)

Hola "María"
```

El uso combinado de comillas simples, dobles o caracteres de escape permite escribir cadenas que incluyen comillas dentro del texto. Esta práctica evita errores de sintaxis y facilita la presentación de mensajes, nombres o expresiones textuales dentro de un programa.

Además de combinar comillas dentro de una cadena, es necesario reconocer cómo Python representa y consulta el texto carácter por carácter. Para ello, conviene tener en cuenta lo siguiente:

- En Python, un carácter individual no es un tipo de dato aparte: se representa como una cadena de longitud uno.
- Las cadenas permiten consultar partes específicas del texto mediante índices y slicing.
- Las cadenas son inmutables, pero permiten generar nuevas cadenas mediante concatenación, repetición, acceso por índice y slicing.

Estas reglas explican por qué es posible extraer una letra, una palabra o un fragmento de una cadena sin alterar el texto original. Python siempre genera una nueva cadena a partir de la que ya existe.

El siguiente código muestra cómo consultar caracteres y obtener fragmentos de una cadena.

Código. Acceso por índice y slicing en cadenas

```
# Acceso por índice
```

```
>>> texto = "Python"
```

```
>>> texto[0]    # 'P'
```

```
>>> texto[-1]   # 'n'
```

```
# Slicing
```

```
>>> texto[0:3]  # 'Pyt'
```

```
>>> texto[2:]    # 'thon'
```

```
>>> texto[::-1]  # 'nohtyP' (invertida)
```

Las cadenas de caracteres incluyen métodos integrados que facilitan la transformación y organización del texto. Estos métodos se aplican escribiendo un punto después de la cadena y luego el nombre de la acción que se desea ejecutar, como convertir a mayúsculas, limpiar espacios, reemplazar palabras, dividir contenido o contar caracteres.

Código. Métodos de cadenas de caracteres

Métodos comunes

```
>>> "hola mundo".upper()    # 'HOLA MUNDO'
```

```
>>> "HOLA MUNDO".lower()    # 'hola mundo'
```

```
>>> "hola mundo".title()    # 'Hola Mundo'
```

```
>>> "  espacios  ".strip()  # 'espacios'
```

```
>>> "hola mundo".replace("mundo", "Python")
```

```
'hola Python'
```

```
>>> "hola,mundo,python".split(",")
```

```
['hola', 'mundo', 'python']
```

```
>>> len("Python")          # 6
```

El uso de métodos de cadena permite preparar textos antes de mostrarlos, compararlos o almacenarlos. Por ejemplo, se pueden limpiar espacios innecesarios,

unificar el uso de mayúsculas y minúsculas, reemplazar palabras o separar información escrita en una misma línea.

La función `len()` devuelve la cantidad de caracteres de una cadena. Esta función es útil para validar datos ingresados por el usuario, como nombres, códigos, contraseñas o mensajes, especialmente cuando se requiere comprobar una longitud mínima o máxima antes de procesar la información.

Una vez reconocidos los tipos de datos básicos y algunas operaciones aplicadas a cadenas de caracteres, es necesario estudiar cómo Python procesa los valores numéricos. Las operaciones aritméticas permiten realizar cálculos, combinar variables y obtener resultados mediante reglas de precedencia que determinan el orden de ejecución de cada expresión.

2.3. Operaciones aritméticas

Las operaciones aritméticas permiten realizar cálculos matemáticos con valores numéricos en Python. Estas operaciones son necesarias para sumar cantidades, restar valores, multiplicar datos, dividir resultados, calcular residuos o elevar números a una potencia dentro de un programa.

En Python, los operadores aritméticos se aplican sobre datos numéricos como `int`, `float` y `complex`. Cuando se combinan tipos diferentes, el lenguaje ajusta el resultado según la jerarquía numérica: al operar un entero con un flotante, el resultado es flotante; al incluir un complejo, el resultado conserva el tipo complejo.

Para comprender el uso de estos operadores, es necesario reconocer qué función cumple cada uno dentro de una expresión matemática:

- **Suma (+):** permite adicionar dos valores numéricos. Por ejemplo, $5 + 3$ devuelve 8, resultado útil para acumular cantidades, calcular totales o actualizar valores dentro de una variable.
- **Resta (-):** permite calcular la diferencia entre dos valores. Por ejemplo, $10 - 4$ devuelve 6, operación frecuente cuando se comparan cantidades, descuentos, saldos o resultados parciales.
- **Multiplicación (*):** permite obtener el producto entre dos valores. Por ejemplo, $4 * 3$ devuelve 12, resultado útil para calcular costos, repeticiones, áreas o cantidades acumuladas.
- **División real (/):** permite dividir dos valores y devuelve un resultado decimal. Por ejemplo, $7 / 2$ devuelve 3.5, incluso cuando los números iniciales son enteros.
- **División entera (//):** permite dividir dos valores y conservar solo la parte entera del resultado. Por ejemplo, $7 // 2$ devuelve 3, descartando los decimales.
- **Módulo (%):** permite obtener el residuo de una división. Por ejemplo, $7 \% 2$ devuelve 1, operación útil para verificar si un número es par o impar.
- **Potencia (**):** permite elevar un número a un exponente. Por ejemplo, $2 ** 3$ devuelve 8, porque representa dos multiplicado por sí mismo tres veces.

El reconocimiento de estos operadores facilita la construcción de expresiones matemáticas claras y evita confusiones al interpretar los resultados. Su uso adecuado permite resolver cálculos básicos, validar operaciones numéricas y preparar datos para procesos posteriores.

El bloque de código permite observar cómo Python ejecuta operaciones aritméticas y cómo cambia el tipo de resultado cuando se combinan valores enteros, flotantes o complejos. Este ejemplo también muestra operadores frecuentes como potencia, división entera y módulo.

Código. Aritmética y jerarquía de tipos numéricos

```
>>> a = 2**3    # 8 (potencia)

>>> c = 31 // 4  # 7 (división entera)

>>> d = 31 % 4   # 3 (módulo)

>>> 1 + 2.0      # 3.0

>>> 2+3j + 5.7    # (7.7+3j)      # 6
```

La secuencia anterior integra operaciones aritméticas frecuentes en Python: potencia (**), división entera (//), módulo (%) y combinación de tipos numéricos. Al operar un entero con un flotante, el resultado se expresa como float; al incluir un número complejo, se conserva la parte real e imaginaria del valor obtenido.

Para aplicar correctamente las operaciones aritméticas en un programa, conviene revisar tres aspectos básicos antes de escribir o ejecutar una expresión:

- a) Identificar el tipo de dato:** verificar si los valores son enteros, flotantes o complejos, ya que el tipo influye en el resultado.
- b) Reconocer el operador adecuado:** seleccionar suma, resta, multiplicación, división, módulo o potencia según el cálculo que se requiere realizar.
- c) Revisar la precedencia:** comprobar el orden en que Python evaluará la expresión, especialmente cuando se combinan varios operadores.

d) Validar el resultado: comparar la salida obtenida con el cálculo esperado para detectar posibles errores de escritura o interpretación.

La comprensión de las operaciones aritméticas permite procesar valores individuales. Con esta base, el estudio de las estructuras de datos compuestas amplía la capacidad del programa para agrupar varios elementos, recorrerlos, organizarlos y transformarlos según las necesidades del proceso.

Después de estudiar las variables, los tipos de datos básicos y las operaciones aritméticas, es necesario avanzar hacia formas más organizadas de almacenar información. En Python, las estructuras de datos compuestas permiten reunir varios valores dentro de una misma variable, lo que facilita representar grupos de elementos, consultar datos relacionados y procesar información de manera más eficiente.

Las estructuras de datos compuestas permiten pasar de datos individuales a conjuntos organizados de información. Su uso adecuado facilita la escritura de programas más claros, flexibles y útiles para resolver situaciones reales.

Una vez comprendido cómo Python procesa operaciones con valores numéricos individuales, es necesario avanzar hacia estructuras que permitan almacenar y organizar varios datos dentro de una misma variable. Este paso facilita el trabajo con listas de elementos, colecciones, rangos y asociaciones clave-valor, que serán abordados en el siguiente apartado.

3. Estructuras de datos compuestas y conversión de tipos

Las estructuras de datos compuestas permiten agrupar varios valores dentro de una sola variable. Esta capacidad es necesaria cuando se manejan listas de elementos, pares clave-valor, secuencias numéricas o colecciones sin elementos repetidos.

El estudio de estas estructuras se complementa con la conversión de tipos, porque en muchos programas es necesario transformar un dato para poder compararlo, calcularlo o presentarlo correctamente. Esta relación resulta especialmente importante cuando los datos provienen de una entrada por teclado.

Además de los tipos básicos, Python dispone de estructuras de datos compuestas que permiten agrupar información relacionada dentro de una misma variable. Estas estructuras son útiles cuando un programa debe manejar varios datos al mismo tiempo, como registros de clientes, listas de productos, turnos de atención, calificaciones, inventarios o resultados de una encuesta.

Para seleccionar una estructura de datos compuesta, es necesario reconocer cómo se comporta la información que se desea almacenar. Esta decisión ayuda a organizar los datos de forma coherente y evita usar estructuras poco adecuadas para el problema que se quiere resolver.

- a. Usar una lista cuando los datos pueden cambiar, agregarse, eliminarse o reorganizarse durante la ejecución del programa.
- b. Usar una tupla cuando los datos deben permanecer estables y no requieren modificación posterior.
- c. Usar un rango cuando se necesita generar una secuencia numérica para conteos, repeticiones o recorridos.

- d. Usar un diccionario cuando cada dato necesita asociarse con una clave que facilite su consulta.
- e. Usar un conjunto cuando se requiere evitar duplicados o comparar grupos de elementos.
- f. Aplicar conversión de tipos cuando los datos recibidos no tienen el formato necesario para calcular, comparar o presentar resultados.

Esta selección permite pasar de valores individuales a colecciones organizadas de información. Con esta base, el siguiente apartado presenta las características y usos de las listas, tuplas, rangos y diccionarios, estructuras fundamentales para almacenar, recorrer y consultar datos en Python.

3.1. Listas, tuplas, rangos y diccionarios

En Python, las listas, las tuplas, los rangos y los diccionarios permiten organizar varios datos dentro de una misma estructura. Su uso facilita almacenar información relacionada, recorrer elementos y construir soluciones más ordenadas que trabajar con variables aisladas.

De acuerdo con la documentación oficial de Python, list, tuple y range hacen parte de los tipos secuencia, mientras que dict corresponde a los tipos de mapeo. Esta diferencia es clave: las secuencias se recorren por posición y los diccionarios recuperan valores mediante claves (Python Software Foundation, s. f.).

Para comprender el uso de listas, tuplas, rangos y diccionarios, se presenta un caso aplicado a una pequeña empresa comercial en Colombia. En este ejemplo, se organiza la información básica de pedidos de una tienda que vende productos de la

canasta familiar y necesita registrar clientes, productos, turnos de atención y datos de ubicación.

- Lista: almacena productos modificables del pedido; permite agregar, eliminar, ordenar y actualizar durante la venta.
- Tupla: conserva datos fijos de ubicación, como coordenadas o códigos que no deben cambiar accidentalmente.
- Rango: genera secuencias numéricas automáticas para turnos, repeticiones o conteos definidos dentro del programa empresarial.
- Diccionario: organiza información mediante claves y valores; facilita consultar clientes, productos o datos asociados rápidamente.

Estas estructuras permiten representar de manera organizada los datos que intervienen en una situación real de programación. En un mismo proceso empresarial, una lista puede guardar productos, una tupla puede conservar datos fijos, un rango puede generar turnos y un diccionario puede reunir información asociada a un cliente.

Para observar esta relación en el lenguaje Python, el siguiente código muestra cómo se pueden declarar y consultar estas estructuras dentro de un mismo caso de uso empresarial.

Código. Uso básico de listas, tuplas, rangos y diccionarios en Python

Caso de uso: registro básico de un pedido en una tienda colombiana

```
productos = ["arroz", "café", "panela"]
```

```
productos.append("aceite")
```

```
ubicacion_sede = (4.7110, -74.0721)
```

```
turnos = range(1, 6)

cliente = {

    "nombre": "Ana",

    "ciudad": "Cali",

    "estado": "activo"

}

print(productos)

print(ubicacion_sede)

print(list(turnos))

print(cliente["nombre"])
```

Este ejemplo muestra que cada estructura cumple una función específica dentro del programa. La lista permite modificar productos, la tupla conserva datos fijos, el rango genera una secuencia de turnos y el diccionario facilita la consulta de información mediante claves.

Cada estructura de datos cumple una función específica: las listas organizan datos modificables, las tuplas conservan valores fijos, los rangos generan secuencias y los diccionarios relacionan claves con valores. Reconocer estas diferencias facilita elegir la estructura adecuada en cada programa.

Después de revisar las estructuras que permiten almacenar datos ordenados o asociados mediante claves, resulta necesario estudiar una estructura orientada a conservar elementos únicos. En Python, los conjuntos facilitan la eliminación de datos

repetidos y la comparación entre grupos de información. Por ejemplo, en una empresa colombiana, pueden usarse para depurar ciudades repetidas en una base de clientes, identificar productos únicos en un inventario o comparar registros de asistencia entre dos jornadas de capacitación.

3.2. Conjuntos

Después de estudiar las listas, las tuplas, los rangos y los diccionarios, resulta necesario reconocer una estructura de datos orientada a un propósito distinto, garantizar que cada elemento aparezca una sola vez dentro de una colección. Esta necesidad surge con frecuencia al procesar información real, donde los datos capturados pueden contener registros duplicados que es preciso depurar antes de analizarlos.

En Python, esta estructura se denomina conjunto (set): una colección no ordenada de elementos únicos, que no admite valores repetidos ni conserva un orden de posición. En una empresa colombiana, un conjunto permite depurar ciudades repetidas en una base de clientes, identificar productos únicos en un inventario o comparar los registros de asistencia entre dos jornadas de capacitación, sin recorrer manualmente cada dato para detectar coincidencias.

Antes de explorar su uso mediante código, conviene precisar los conceptos y las operaciones fundamentales que caracterizan a los conjuntos en Python. Cada definición incluye un ejemplo breve que puede copiarse directamente en Visual Studio Code para observar el resultado:

Conjunto (set): colección no ordenada de elementos únicos, definida con la función `set()` o con llaves `{ }`. No permite valores duplicados ni almacena los elementos en un orden fijo de posición. Por ejemplo:

VS Code

```
colores = {'rojo', 'azul', 'verde', 'azul'}  
  
print(colores)
```

Al ejecutarlo en VS Code, el resultado es `{'rojo', 'azul', 'verde'}`: el valor repetido se elimina automáticamente, porque un conjunto nunca conserva duplicados.

- **Unión (`|`):** operación que combina dos conjuntos en uno solo, incluyendo todos los elementos presentes en cualquiera de los dos conjuntos originales, sin repetir ningún valor en el resultado. Por ejemplo:

VS Code

```
a = {1, 2, 3}  
  
b = {3, 4, 5}  
  
print(a | b)
```

Al ejecutarlo en VS Code, el resultado es `{1, 2, 3, 4, 5}`, reuniendo los elementos de ambos conjuntos sin duplicar el 3.

- **Intersección (`&`):** operación que devuelve únicamente los elementos que se repiten en ambos conjuntos al mismo tiempo; si no existe ningún valor en común, el resultado es un conjunto vacío. Por ejemplo:

VS Code

```
a = {1, 2, 3}
```

```
b = {3, 4, 5}
```

```
print(a & b)
```

Al ejecutarlo en VS Code, el resultado es {3}, el único valor presente en los dos conjuntos.

- **Diferencia (-):** operación que retorna los elementos que pertenecen únicamente al primer conjunto y no al segundo; el orden de los conjuntos dentro de la operación cambia el resultado. Por ejemplo:

VS Code

```
a = {1, 2, 3}
```

```
b = {3, 4, 5}
```

```
print(a - b)
```

Al ejecutarlo en VS Code, el resultado es {1, 2}, los valores exclusivos de a.

- **Diferencia simétrica (^):** operación que devuelve los elementos presentes en alguno de los dos conjuntos, pero excluye los valores que se repiten de forma simultánea en ambos. Por ejemplo:

VS Code

```
a = {1, 2, 3}
```

```
b = {3, 4, 5}
```

```
print(a ^ b)
```

Al ejecutarlo en VS Code, el resultado es {1, 2, 4, 5}, todos los valores no compartidos.

- **Método add():** instrucción que agrega un nuevo elemento a un conjunto ya existente; si el valor ya está presente en el conjunto, Python no genera ningún error ni lo duplica. Por ejemplo:

VS Code

```
frutas = {'mango', 'pera'}
```

```
frutas.add('limón')
```

```
print(frutas)
```

Al ejecutarlo en VS Code, el resultado es {'mango', 'pera', 'limón'}, con el nuevo elemento incorporado.

- **Método discard():** instrucción que elimina un elemento específico de un conjunto de forma segura; a diferencia del método remove(), no genera ningún error si el valor buscado no existe. Por ejemplo:

VS Code

```
frutas = {'mango', 'pera'}
```

```
frutas.discard('uva')
```

```
print(frutas)
```

Al ejecutarlo en VS Code, el resultado es {'mango', 'pera'}, sin ningún error, aunque 'uva' no estaba presente.

Estas definiciones, ya verificadas en código, permiten comprender el propósito de cada operación antes de aplicarla en los ejemplos integrados que se presentan a continuación: mientras que la creación y las operaciones matemáticas manipulan el conjunto como un todo, los métodos `add()` y `discard()` permiten modificar sus elementos de forma individual.

Además de las operaciones y los métodos que modifican un conjunto, Python ofrece dos métodos que permiten comparar la relación entre dos conjuntos sin alterarlos: `issubset()`, que verifica si todos los elementos de un conjunto están contenidos en otro, e `issuperset()`, que verifica la relación inversa, es decir, si un conjunto contiene todos los elementos del otro. Ambos métodos devuelven un valor booleano y son complementarios: si A es subconjunto de B, entonces B es, necesariamente, superconjunto de A.

El siguiente código aplica ambos métodos sobre los mismos dos conjuntos, evidenciando esta relación complementaria.

Código. Relación entre conjuntos con `issubset()` e `issuperset()`

```
primarios = {'rojo', 'azul', 'amarillo'}  
  
muestra = {'rojo', 'azul'}  
  
print(muestra.issubset(primarios))  
  
print(primarios.issuperset(muestra))
```


Resultado esperado

True

True

El resultado anterior confirma la relación descrita: como muestra es subconjunto de primarios, se cumple automáticamente que primarios es superconjunto de muestra. Reconocer esta relación resulta útil en la práctica, ya que permite inferir una condición a partir de la otra sin necesidad de comprobarlas de forma independiente, lo que agiliza la verificación de datos cuando se trabaja con grupos de información relacionados entre sí.

El siguiente código ejecuta en un solo bloque las cuatro operaciones entre conjuntos que se presentaron anteriormente, lo que permite comparar directamente el resultado que produce cada operador sobre el mismo par de valores. Este tipo de comparación conjunta resulta útil, por ejemplo, al cruzar los conjuntos de clientes de la sede Bogotá y la sede Medellín para responder distintas preguntas de análisis sin necesidad de reescribir los datos en cada caso.

Código. Operaciones matemáticas de conjuntos

```
A = {1, 2, 3, 4}
```

```
B = {3, 4, 5, 6}
```

```
print(A | B) # Unión
```

```
print(A & B) # Intersección
```

```
print(A - B) # Diferencia
```

```
print(A ^ B) # Diferencia simétrica
```

Resultado esperado

```
{1, 2, 3, 4, 5, 6}
```

```
{3, 4}
```

```
{1, 2}
```

```
{1, 2, 5, 6}
```

Al ejecutar las operaciones entre conjuntos, se observa que cada operador cumple una función específica: la unión reúne todos los elementos, la intersección conserva los comunes, la diferencia muestra los exclusivos de un conjunto y la diferencia simétrica identifica los valores no compartidos. Comparar estos resultados facilita comprender cómo se relacionan los datos y qué información aporta cada operación.

Para consultar estas operaciones sin necesidad de ejecutar código cada vez, la siguiente tabla las organiza en un formato de referencia rápida, con un ejemplo de uso aplicado a una empresa colombiana con varias sucursales.

Tabla 4. Operaciones de conjuntos en Python

Operación	Descripción	Ejemplo de uso
A B (unión)	Todos los elementos de ambos conjuntos.	Reunir los clientes registrados en la sede Bogotá y en la sede Medellín.
A y B (intersección)	Solo los elementos comunes a ambos.	Identificar los clientes que compraron en ambas sedes.

Operación	Descripción	Ejemplo de uso
$A - B$ (diferencia)	Elementos de A que no están en B.	Encuentre los productos que solo se ofrecen en la sede Bogotá.
$A \Delta B$ (diferencia simétrica)	Elementos en A o en B, pero no en ambos.	Detectar los clientes que solo compraron en una de las dos sedes.

Reconocer qué pregunta responde cada operación evita aplicar la fórmula equivocada al analizar datos reales, la unión reúne, la intersección aísla lo común, la diferencia identifica lo exclusivo de un grupo y la diferencia simétrica resalta lo que no comparten entre sí.

Además de las operaciones matemáticas entre conjuntos, Python dispone de métodos que permiten modificar sus elementos. El método `add()` agrega un valor nuevo; `discard()` elimina un elemento sin generar error cuando el dato no existe, y `clear()` vacía el conjunto por completo. Estas acciones son útiles cuando se necesita actualizar información sin permitir valores duplicados.

El siguiente código muestra cómo agregar y eliminar elementos de un conjunto. Para visualizar el resultado en Visual Studio Code, se utiliza la función `print()`, ya que las instrucciones por sí solas no siempre muestran la salida cuando se ejecutan desde un archivo `.py`.

Código. Actualización de elementos en un conjunto

```
# Métodos para modificar conjuntos

colores = {"rojo", "verde", "azul"}

colores.add("amarillo")    # Agrega un nuevo elemento.
```

```
colores.discard("negro")    # No genera error, aunque el elemento no exista.

colores.discard("rojo")    # Elimina el elemento indicado.

print(colores)
```

Resultado esperado

```
{'verde', 'amarillo', 'azul'}
```

El resultado puede mostrarse en un orden diferente, porque los conjuntos no conservan una posición fija para sus elementos. Lo importante es verificar que el valor "amarillo" fue agregado, que "rojo" fue eliminado y que "negro" no generó error al no estar presente en el conjunto.

Además de modificar elementos, los conjuntos permiten verificar relaciones entre grupos de datos. Los métodos `issubset()` e `issuperset()` ayudan a comprobar si un conjunto está contenido dentro de otro o si contiene todos los elementos de otro conjunto.

Para consolidar el estudio de las estructuras de datos compuestas, la Tabla 6 compara las características principales de listas, tuplas, conjuntos y diccionarios. Esta referencia permite identificar sus diferencias en sintaxis, orden, mutabilidad, manejo de duplicados, forma de acceso y uso frecuente dentro de un programa en Python.

Tabla 5. Comparación de estructuras de datos compuestas en Python

Importancia	Lista (list)	Tupla (tuple)	Conjunto (set)	Diccionario (dict)
Sintaxis	[]	()	{ } o set()	{clave: valor}

Importancia	Lista (list)	Tupla (tuple)	Conjunto (set)	Diccionario (dict)
Orden	Conserva el orden.	Conserva el orden.	No tiene orden posicional fijo.	Conserva el orden de inserción desde Python 3.7.
Mutabilidad	Se puede modificar.	No se puede modificar.	Se puede modificar.	Se puede modificar.
Duplicados	Permite valores repetidos.	Permite valores repetidos.	No permite valores repetidos.	No permite claves repetidas.
Forma de acceso	Por índice.	Por índice.	Por recorrido o pertenencia.	Por clave.
Uso típico	Colecciones que cambian.	Datos fijos.	Elementos únicos.	Pares clave-valor.

Esta comparación facilita la selección de la estructura más adecuada según la necesidad del programa. Las listas son útiles cuando los datos deben cambiar; las tuplas, cuando se requiere conservar información fija; los conjuntos, cuando se necesita evitar duplicados, y los diccionarios, cuando la información debe consultarse mediante claves.

Una vez reconocidas las estructuras de datos compuestas, es frecuente que los valores deban transformarse antes de calcularlos, compararlos o mostrarlos. Por esta razón, el siguiente apartado aborda la conversión de tipos de datos, un proceso necesario para adaptar la información al formato requerido por cada operación en Python.

3.3. Conversión de tipos de datos

La conversión de tipos de datos permite transformar un valor de un tipo a otro para que pueda utilizarse correctamente dentro de una operación. En Python, esta acción es frecuente cuando se reciben datos como texto, se realizan cálculos numéricos, se comparan valores o se preparan resultados para mostrarlos en pantalla.

Esta conversión debe aplicarse de manera intencional, porque no todos los valores pueden transformarse a cualquier tipo. Por ejemplo, la cadena "25" puede convertirse en número entero con `int("25")`, pero la cadena "hola" no puede convertirse en número, ya que no representa un valor numérico válido.

Para comprender las funciones de conversión más utilizadas en Python, se presentan las siguientes definiciones con ejemplos básicos:

A. Funciones de conversión en Python

- **`int()`**

Convierte un valor válido en número entero. Por ejemplo, `int("25")` transforma la cadena "25" en el número 25, lo que permite realizar operaciones como sumar, restar o comparar edades, cantidades o conteos.

- **`float()`**

Convierte un valor válido en número decimal. Por ejemplo, `float("18.66")` transforma el texto "18.66" en un valor numérico con decimales, útil para precios, promedios, porcentajes o medidas.

- **str()**

Convierte un valor en cadena de caracteres. Por ejemplo, `str(35)` transforma el número 35 en texto, lo que permite unirlo con mensajes o mostrarlo dentro de una salida informativa.

- **bool()**

Convierte un valor a su equivalente lógico. Por ejemplo, `bool(0)` devuelve `False`, mientras que `bool("hola")` devuelve `True`. Esta función es útil para validar si un dato está vacío o contiene información.

- **list(), tuple() y set()**

Convierten secuencias en listas, tuplas o conjuntos. Por ejemplo, `list("Python")` separa cada carácter en una lista, mientras que `set([1, 1, 2])` elimina valores repetidos.

- **ord() y chr()**

Permiten trabajar con códigos Unicode. Por ejemplo, `ord("A")` devuelve 65, y `chr(65)` devuelve "A". Estas funciones son útiles cuando se requiere relacionar caracteres con su código numérico.

Estas funciones facilitan la adaptación de los datos según la necesidad del programa. Su uso adecuado evita errores al calcular, comparar, validar o presentar información, especialmente cuando los valores provienen de entradas externas o formularios.

B. Código de ejemplo

El siguiente código muestra cómo convertir datos recibidos como texto en valores numéricos y, posteriormente, transformar un resultado numérico en cadena de caracteres. Este proceso es común cuando se capturan datos desde teclado o desde formularios.

Código. Conversión básica con `int()`, `float()` y `str()`

```
edad_texto = "25"

edad = int(edad_texto)

edad_futura = edad + 10

precio_texto = "18.66"

precio = float(precio_texto)

mensaje = "Edad futura: " + str(edad_futura)

print(edad_futura)

print(precio)

print(mensaje)
```

Resultado esperado

35

18.66

Edad futura: 35

El código permite observar que la edad, inicialmente almacenada como texto, se convierte en número entero para realizar una suma. Luego, el resultado se transforma en cadena de caracteres para integrarlo dentro de un mensaje.

La siguiente tabla reúne funciones de conversión y utilidad frecuentes en Python. Este recurso sirve como referencia rápida para reconocer qué función aplicar según el tipo de dato que se necesita obtener.

Tabla 6. Funciones de conversión de tipos en Python

Función	Descripción	Ejemplo de uso
<code>int(x)</code>	Convertir x un número entero. Si x es así float, elimina la parte decimal.	<code>int("25") → 25</code>
<code>float(x)</code>	Convierta x un número de punto flotante.	<code>float("18.66") → 18.66</code>
<code>str(x)</code>	Convertir x una cadena de caracteres.	<code>str(35) → "35"</code>
<code>bool(x)</code>	Convierte x un valor booleano: True o False.	<code>bool("") → Falso</code>
<code>list(x)</code>	Convierte x a lista.	<code>list("Py") → ["P", "y"]</code>
<code>tuple(x)</code>	Convierte x a tupla.	<code>tuple([1, 2]) → (1, 2)</code>
<code>set(x)</code>	Convierte x un conjunto y elimina duplicados.	<code>set([1, 1, 2]) → {1, 2}</code>
<code>ord(c)</code>	Devuelve el código Unicode del carácter c.	<code>ord("A") → 65</code>
<code>chr(n)</code>	Devuelve el carácter correspondiente al código n.	<code>chr(65) → "A"</code>

El reconocimiento de estas funciones permite seleccionar la conversión más adecuada según la operación que se desea realizar. Así, se reducen errores al procesar entradas de usuario, transformar secuencias, depurar duplicados o preparar datos para su presentación.

C. Código de aplicación

El siguiente código aplica varias funciones de conversión y utilidad sobre valores simples y secuencias. Este ejemplo permite comparar cómo cambia la representación de un dato según la función utilizada.

Código. Conversión de tipos y funciones auxiliares

```
print(bool(0))  
  
print(bool("hola"))  
  
print(list("Python"))  
  
print(tuple([1, 2, 3]))  
  
print(set([1, 1, 2, 3]))  
  
print(ord("A"))  
  
print(chr(65))
```

Resultado esperado

False

True

`['P', 'y', 't', 'h', 'o', 'n']`

`(1, 2, 3)`

`{1, 2, 3}`

`65`

`A`

El resultado muestra que `bool()` interpreta valores vacíos o cero como `False`, mientras que los valores con contenido se interpretan como `True`. También se observa que `list()`, `tuple()` y `set()` cambian la forma de organizar una secuencia, y que `ord()` y `chr()` permiten relacionar caracteres con sus códigos Unicode.

En síntesis, la conversión de tipos es una práctica necesaria para asegurar que cada dato tenga el formato adecuado antes de ser procesado. Esta habilidad resulta especialmente importante cuando los datos provienen de una entrada por teclado, de un formulario o de una fuente externa.

Con esta base, el siguiente tema aborda la entrada y salida de datos, proceso mediante el cual un programa recibe información del usuario, la procesa y presenta resultados. Esta relación es fundamental, porque los datos capturados con `input()` llegan como texto y, en muchos casos, deben convertirse antes de realizar cálculos, comparaciones o validaciones.

4. Entrada y salida de datos

La entrada y la salida de datos permiten que un programa interactúe con la persona que lo utiliza. A través de `input()`, Python captura información durante la ejecución; mediante `print()`, presenta mensajes, resultados y datos procesados en la consola.

Para comprender este proceso, es necesario diferenciar las operaciones que permiten recibir información, mostrar resultados, convertir valores y organizar la salida. Estos conceptos fortalecen la construcción de programas secuenciales claros, verificables y fáciles de interpretar.

A continuación, se presentan los conceptos básicos que orientan la entrada y salida de datos en Python:

- **Entrada de datos:** es el proceso mediante el cual un programa recibe información durante su ejecución. En Python se realiza con `input()`, función que captura texto y permite iniciar cálculos, decisiones o mensajes personalizados según la necesidad planteada.
- **Salida de datos:** es la operación que permite mostrar al usuario los resultados producidos por el programa. En Python se usa `print()`, función que presenta textos, números y mensajes formateados en consola de forma clara durante la ejecución.
- **Conversión de tipos:** consiste en transformar un dato de un tipo a otro para usarlo correctamente en operaciones. Como `input()` siempre devuelve texto, funciones como `int()` y `float()` permiten convertir valores antes de calcular o comparar información ingresada.

- **Formato de salida:** hace referencia a la forma en que se presentan los resultados para facilitar su lectura. En Python pueden emplearse f-strings, `format()` o separadores para organizar valores, unidades y mensajes comprensibles en consola, durante la ejecución.

Estos conceptos permiten reconocer que la interacción básica con el usuario no termina al capturar un dato: también exige validar su tipo, procesarlo de manera adecuada y presentar una respuesta comprensible. Con esta base, se aborda la función `input()` como primer mecanismo de entrada estándar en Python.

Capturar un dato es apenas el primer paso. También es necesario validarlo, procesarlo y presentarlo de forma comprensible para el usuario. El siguiente apartado profundiza en la función `input()`, como el primer mecanismo de entrada estándar en Python.

4.1. Entradas estándar: concepto y función `input()`

La entrada estándar permite que un programa reciba información durante su ejecución. En Python, este proceso se realiza mediante la función `input()`, la cual muestra un mensaje orientador en pantalla y espera que se ingrese un dato desde el teclado.

El valor capturado con `input()` siempre se recibe como una cadena de caracteres (`str`), aunque se escriba un número. Por esta razón, cuando el dato se necesita para realizar cálculos, debe convertirse al tipo correspondiente mediante funciones como `int()` o `float()`.

Para comprender el uso de la entrada estándar en Python, se deben tener en cuenta los siguientes aspectos:

- **Captura de texto:** la función `input()` permite recibir nombres, ciudades, códigos o mensajes escritos por el usuario. Por ejemplo, `nombre = input("Ingrese su nombre: ")` guarda el dato como texto y permite mostrarlo posteriormente con `print(nombre)`.
- **Conversión a entero:** cuando se necesita trabajar con edades, cantidades o conteos, el valor capturado debe convertirse con `int()`. Por ejemplo, `edad = int(input("Ingrese su edad: "))` permite usar la edad en operaciones numéricas.
- **Conversión a flotante:** cuando se capturan datos con decimales, como estaturas, precios, promedios o medidas, se utiliza `float()`. Por ejemplo, `estatura = float(input("Ingrese su estatura: "))` convierte el dato ingresado en un número decimal.
- **Validación de entrada:** si se ingresa un valor que no puede convertirse al tipo esperado, Python genera un error. Por ejemplo, al escribir letras en una entrada que espera un número, puede generarse un `ValueError`.

Estas acciones permiten construir programas interactivos que solicitan datos, los transforman y los preparan para realizar operaciones. El uso adecuado de `input()` fortalece la lógica secuencial del programa, porque permite pasar de datos fijos escritos en el código a información ingresada durante la ejecución.

A. Código de captura de datos con input()

El siguiente código muestra cómo capturar un nombre, una edad y una estatura. Este ejemplo permite observar que el nombre se conserva como texto, mientras que la edad y la estatura se convierten a tipos numéricos para poder procesarse correctamente.

Código. Captura de texto y conversión de datos con input()

```
# Entrada básica de datos

nombre = input("Ingrese su nombre: ")

print(nombre)

print(type(nombre))

edad = int(input("Ingrese su edad: "))

print(edad)

print(type(edad))

estatura = float(input("Ingrese su estatura: "))

print(estatura)

print(type(estatura))
```

Resultado esperado

Ingrese su nombre: Maria

Maria

```
<class 'str'>
```

```
Ingrese su edad: 25
```

```
25
```

```
<class 'int'>
```

```
Ingrese su estatura: 1.75
```

```
1.75
```

```
<class 'float'>
```

El resultado permite verificar que Python reconoce cada dato según la conversión aplicada. El nombre queda como str, la edad se transforma en int y la estatura se convierte en float. Esta diferencia es importante porque cada tipo de dato permite realizar operaciones distintas.

B. Código. Programa secuencial para calcular un promedio

La función input() también se puede utilizar dentro de programas secuenciales que reciben varios datos, realizan cálculos y muestran un resultado. En el siguiente caso, se capturan tres notas, se convierten a valores decimales y se calcula el promedio académico.

Código. Programa secuencial: cálculo de promedio

```
# Programa: calcular promedio de tres notas
```

```
nota1 = float(input("Ingrese la primera nota: "))
```

```
nota2 = float(input("Ingrese la segunda nota: "))
```



```
nota3 = float(input("Ingrese la tercera nota: "))  
  
promedio = (nota1 + nota2 + nota3) / 3  
  
print(f"El promedio es: {promedio:.1f}")
```

Resultado esperado

Ingrese la primera nota: 4.5

Ingrese la segunda nota: 3.8

Ingrese la tercera nota: 4.2

El promedio es: 4.2

Este ejemplo integra entrada de datos, conversión de tipos, operación aritmética y salida formateada. La expresión `f"El promedio es: {promedio:.1f}"` permite presentar el resultado con una cifra decimal, lo que mejora la claridad del mensaje mostrado en pantalla.

C. Código de programa secuencial: registro académico

El siguiente ejemplo integra varios tipos de datos en un mismo programa. Se captura información de un estudiante, se calculan notas ponderadas y se determina si aprueba según el promedio obtenido. Este caso permite observar cómo `input()`, `float()`, operadores aritméticos y operadores de comparación trabajan de forma conjunta.

Código. Registro académico

```
# Programa: registro académico de un estudiante
```

```
print("=== Registro Académico ===")

nombre = input("Nombre del estudiante: ")

codigo = input("Código del estudiante: ")

nota1 = float(input("Nota primer corte (0 a 5): "))

nota2 = float(input("Nota segundo corte (0 a 5): "))

nota3 = float(input("Nota tercer corte (0 a 5): "))

promedio = (nota1 * 0.30) + (nota2 * 0.30) + (nota3 * 0.40)

aprueba = promedio >= 3.0

print()

print(f"Estudiante: {nombre}")

print(f"Código: {codigo}")

print(f"Nota 1: {nota1:.1f} (30 %)")

print(f"Nota 2: {nota2:.1f} (30 %)")

print(f"Nota 3: {nota3:.1f} (40 %)")

print(f"Promedio: {promedio:.2f}")

print(f"Aprueba: {aprueba}")
```

Resultado esperado

```
=== Registro Académico ===
```

Nombre del estudiante: Ana

Código del estudiante: A001

Nota primer corte (0 a 5): 4.0

Nota segundo corte (0 a 5): 3.5

Nota tercer corte (0 a 5): 4.2

Estudiante: Ana

Código: A001

Nota 1: 4.0 (30 %)

Nota 2: 3.5 (30 %)

Nota 3: 4.2 (40 %)

Promedio: 3.93

El ejemplo anterior permite reconocer cómo diferentes tipos de datos pueden coexistir en un mismo programa. El nombre y el código se trabajan como cadenas de caracteres (str), las notas y el promedio como números decimales (float), y la aprobación como un valor booleano (bool).

La función `input()` permite construir programas más cercanos a situaciones reales, porque recibe información ingresada durante la ejecución. Sin embargo, como todo dato capturado llega inicialmente como texto, es necesario convertirlo al tipo adecuado antes de calcular, comparar o validar información.

Una vez capturados y procesados los datos, el programa debe presentar resultados claros para quien lo utiliza. Por ello, el siguiente apartado aborda la salida

estándar mediante la función `print()` y algunas técnicas de formato que permiten mostrar mensajes, valores y resultados de manera ordenada y comprensible.

4.2. Salidas estándar: función `print()` y técnicas de formato

La salida estándar permite que un programa comunique información a la persona que lo utiliza. En Python, esta acción se realiza principalmente con la función `print()`, que muestra en pantalla mensajes, valores almacenados en variables, resultados de cálculos o textos organizados con formato.

Para utilizar correctamente la salida estándar en Python, se deben reconocer los siguientes elementos:

- a) **Función `print()`:** permite mostrar información en la consola. Puede recibir textos, números, variables o resultados de operaciones.
- b) **Parámetro `sep`:** permite definir el separador entre varios valores mostrados por `print()`. Por defecto, Python usa un espacio.
- c) **Parámetro `end`:** permite definir cómo termina la salida. Por defecto, Python agrega un salto de línea al final.
- d) **F-strings:** permiten insertar variables y expresiones dentro de una cadena de texto mediante llaves `{}`. Son recomendadas por su legibilidad.
- e) **Método `format()`:** permite insertar valores dentro de una cadena usando marcadores `{}`. Sigue siendo válido y útil en diferentes versiones de Python.
- f) **Operador `%`:** permite dar formato a cadenas mediante especificadores como `%s`, `%d` o `%.2f`. Aunque sigue funcionando, se recomienda priorizar las f-strings en código moderno.

Estos elementos ayudan a presentar resultados de forma ordenada y comprensible. Su uso es importante cuando el programa debe mostrar nombres, fechas, valores monetarios, promedios, porcentajes o mensajes dirigidos al usuario.

A. Uso básico de print()

La función `print()` puede mostrar uno o varios valores en pantalla. Cuando recibe varios argumentos separados por comas, Python los imprime con un espacio entre ellos. También es posible modificar ese comportamiento con los parámetros `sep` y `end`.

Código. Uso básico de print() con múltiples argumentos

```
# Uso básico de print()

print("Hola", "Mundo")

# Personalizar separador

print("A", "B", "C", sep="-")

# Mostrar una fecha con separador personalizado

print("08", "07", "2026", sep="/")

# Personalizar fin de línea

print("Línea 1", end=" | ")

print("Línea 2")
```

Resultado esperado

Hola Mundo

A-B-C

08/07/2026

Línea 1 | Línea 2

Este código muestra que `print()` no solo sirve para imprimir mensajes, sino también para organizar la salida. El parámetro `sep` define cómo se separan los valores, mientras que `end` controla el cierre de la línea y permite continuar la salida en la misma fila.

Las f-strings permiten construir mensajes claros al combinar texto, variables y expresiones dentro de una misma salida. Esta técnica facilita la lectura del código y mejora la presentación de los resultados, porque evita concatenaciones extensas y permite controlar la forma en que se muestran los datos. En Python, su uso es frecuente en tres situaciones principales:

a) Formato de salida con f-strings

Las f-strings permiten construir mensajes claros combinando texto y variables. Para utilizarlas, se antepone la letra `f` antes de la cadena y se escriben las variables o expresiones entre llaves `{}`. Esta técnica es recomendada por su claridad y facilidad de lectura.

Código. Formato de salida con f-strings

```
nombre = "María"
```

```
edad = 25
```

```
precio = 49.999
```

```
print(f"{nombre} tiene {edad} años.")
```

```
print(f"Precio: ${precio:.2f}")
```

```
print(f"{'Python':>15}")
```

Resultado esperado

María tiene 25 años.

Precio: \$50.00

Python

El resultado muestra tres usos frecuentes de las f-strings: insertar variables en un mensaje, presentar un número con dos decimales y alinear texto hacia la derecha. Esta técnica es útil para construir salidas claras en reportes, facturas, promedios o resúmenes de información.

b) Formato de salida con el método format()

El método format() permite insertar valores dentro de una cadena mediante marcadores representados con llaves {}. Los valores se ubican en el orden en que aparecen dentro del método, aunque también pueden numerarse para reutilizarlos o reorganizarlos dentro del mensaje.

Código. Formato de salida con el método format()

```
nombre = "María"
```

```
edad = 25
```

```
print("{} tiene {} años.".format(nombre, edad))
```

```
print("{1} tiene {0} años.".format(edad, nombre))
```

Resultado esperado

María tiene 25 años.

María tiene 25 años.

En este ejemplo, el método `format()` permite insertar datos dentro de una cadena de texto. En la primera línea, los valores se ubican en el mismo orden en que aparecen dentro del método. En la segunda línea, se usan posiciones numéricas: `{0}` representa la edad y `{1}` representa el nombre. Por eso, aunque los datos se escriben como `format(edad, nombre)`, la salida final sigue siendo: María tiene 25 años.

c) Formato de salida con el operador %

El operador `%` corresponde a un estilo clásico de formato. Utiliza especificadores como `%s` para cadenas, `%d` para enteros y `%.2f` para números decimales con dos cifras. Aunque sigue siendo válido, se recomienda utilizarlo principalmente para comprender o mantener código existente.

Código. Formato de salida con el operador %

```
nombre = "María"
```

```
edad = 25
```

```
print("%s tiene %d años." % (nombre, edad))
```

```
print("Valor: %.2f" % 3.14159)
```


Resultado esperado

María tiene 25 años.

Valor: 3.14

El resultado muestra cómo el operador % reemplaza los especificadores por los valores indicados después de la cadena. Esta técnica permite controlar el tipo de dato y la cantidad de decimales, aunque las f-strings ofrecen una escritura más clara para programas actuales.

Para comparar los principales especificadores de formato, la siguiente tabla reúne los símbolos más utilizados, el tipo de dato al que se aplican y un ejemplo de salida. Esta referencia permite identificar cómo presentar cadenas, enteros y números decimales con mayor precisión.

Tabla 7. Formatos para la salida de datos en Python

Especificador	Tipo de dato	Ejemplo	Resultado esperado
%s	Cadena de caracteres	"%s" % "Hola"	Hola
%d	Número entero	"%d" % 25	25
%f	Número decimal	"%f" % 3.141593	3.141593
%.2f	Número decimal con dos cifras	"%.2f" % 3.141593	3.14
%10d	Número entero alineado a la derecha	"%10d" % 25	25
{:>10}	Texto alineado a la derecha	"{:>10}".format("Python")	Python

Especificador	Tipo de dato	Ejemplo	Resultado esperado
<code>{:.2f}</code>	Decimal con dos cifras en <code>format()</code>	<code>" {:.2f}".format(49.999)</code>	50.00
<code>{precio:.2f}</code>	Decimal con dos cifras en f-string	<code>f" {precio:.2f}"</code>	50.00

El uso de estos formatos permite presentar la información de manera clara, ordenada y precisa. Esta práctica es útil cuando se muestran valores monetarios, promedios, porcentajes, fechas, códigos o resultados que requieren una salida uniforme y fácil de interpretar.

Para presentar resultados claros en Python, no basta con mostrar los datos en pantalla. También es necesario definir cómo se organizarán los mensajes, cuántos decimales se mostrarán, qué etiquetas acompañarán los valores y qué técnica de formato será más adecuada para facilitar su lectura.

- **Usar f-strings como técnica principal:** se recomienda emplearlas en programas actuales por su claridad y facilidad de lectura. Permiten insertar variables, expresiones y formatos dentro de una misma cadena sin construir mensajes extensos o difíciles de interpretar.
- **Definir la cantidad de decimales:** cuando se presentan precios, promedios, porcentajes o medidas, es conveniente indicar cuántas cifras decimales deben mostrarse. Por ejemplo, `{precio:.2f}` permite presentar un valor con dos decimales.

- **Incluir etiquetas claras:** cada resultado debe ir acompañado de una palabra o frase que explique su significado. Por ejemplo, escribir Promedio: 4.25 es más claro que mostrar únicamente el número 4.25.
- **Cuidar unidades y símbolos:** cuando el resultado corresponde a dinero, porcentaje, medida o cantidad, debe presentarse con la unidad adecuada. Esta práctica evita confusiones y mejora la interpretación de la información.
- **Organizar la salida visualmente:** la alineación, los espacios y los saltos de línea ayudan a que los resultados sean más fáciles de leer. Esta organización es útil en reportes, facturas, listados o resúmenes generados por el programa.

Aplicar estas recomendaciones permite que la salida del programa sea comprensible, precisa y útil para quien la interpreta. Una presentación adecuada de los resultados fortalece la calidad del código y reduce errores de lectura, especialmente cuando se trabajan datos numéricos o información destinada a la toma de decisiones.

La salida de datos debe comunicar resultados de forma clara, precisa y ordenada. Un programa que presenta etiquetas comprensibles, unidades correctas y decimales adecuados facilita la interpretación de la información y demuestra buenas prácticas de programación.

El manejo de la entrada y la salida de datos permite que el programa reciba información, la procese y presente resultados comprensibles. A partir de esta base, el estudio de los operadores y las funciones integradas amplía las posibilidades de procesamiento, porque permite calcular, comparar, validar, transformar datos y construir instrucciones más completas dentro de un programa secuencial en Python.

5. Operadores y funciones integradas

Los operadores y las funciones integradas permiten transformar datos en resultados útiles dentro de un programa. Con estos recursos es posible calcular valores, comparar condiciones, validar información, convertir tipos de datos y recorrer secuencias sin construir instrucciones adicionales desde cero.

Este tema integra los aprendizajes previos sobre variables, tipos de datos, estructuras compuestas, entrada y salida. Las variables almacenan valores, los tipos definen qué operaciones son válidas y los operadores o funciones permiten procesar la información de manera ordenada.

Para facilitar la comprensión, se presentan las principales categorías que intervienen en el procesamiento de datos en Python:

- **Operadores aritméticos:** realizan cálculos matemáticos entre valores numéricos, como suma, resta, multiplicación, división, módulo y potencia. Su aplicación permite resolver operaciones básicas y expresar fórmulas dentro de programas secuenciales.
- **Operadores de asignación:** almacenan o actualizan valores en una variable. Además de la asignación simple, permiten abreviar operaciones frecuentes, como sumar y guardar el resultado en la misma variable durante la ejecución del programa.
- **Operadores relacionales:** comparan dos valores y devuelven un resultado booleano: verdadero o falso. Son útiles para verificar igualdad, diferencia, mayor que, menor que y otras condiciones necesarias en validaciones.

- **Operadores lógicos:** combinan expresiones booleanas mediante and, or y not. Facilitan la construcción de condiciones más completas, especialmente cuando una decisión depende de varios criterios evaluados al mismo tiempo.
- **Operadores de cadena:** permiten manipular texto mediante concatenación, repetición y comparación. Su uso ayuda a construir mensajes, validar entradas y presentar información textual de forma clara para quien ejecuta el programa.
- **Funciones integradas:** son herramientas disponibles directamente en Python para realizar tareas comunes, como mostrar datos, convertir tipos, calcular longitudes, ordenar valores o explorar objetos, sin importar módulos externos.

Estas categorías permiten comprender cómo Python procesa los datos después de recibirlos y antes de presentarlos como salida. A partir de esta base, se profundiza en los operadores aritméticos y en la precedencia, ya que estos determinan el orden en que se resuelven las expresiones numéricas dentro de un programa.

5.1. Operadores aritméticos y precedencia

Los operadores aritméticos permiten realizar cálculos matemáticos con valores numéricos. En Python se utilizan para sumar, restar, multiplicar, dividir, calcular potencias y obtener residuos de una división. Su uso es frecuente en programas que procesan precios, cantidades, promedios, porcentajes, medidas o cualquier información que requiera operaciones numéricas.

Además de conocer el símbolo de cada operador, es necesario comprender la precedencia. Esta regla indica el orden en que Python evalúa una expresión cuando contiene varios operadores. Por ejemplo, en una operación combinada, primero se calculan las potencias, luego multiplicaciones, divisiones y módulos, y finalmente sumas y restas. Los paréntesis permiten modificar ese orden para hacer más clara la operación.

Para interpretar correctamente las expresiones aritméticas, conviene tener presentes estos aspectos:

- a. Identificar el operador utilizado:** permite reconocer si la operación corresponde a suma, resta, multiplicación, división, módulo o potencia.
- b. Revisar el tipo de división:** el operador `/` devuelve una división real, mientras que `//` devuelve una división entera, descartando la parte decimal.
- c. Aplicar la precedencia:** Python evalúa primero los operadores de mayor prioridad. Cuando se requiere alterar el orden natural, se emplean paréntesis.
- d. Validar el resultado:** comparar la salida obtenida con el cálculo esperado permite detectar errores de escritura, interpretación o lógica.

Estos criterios ayudan a escribir expresiones matemáticas más claras y reducen errores frecuentes al combinar varios operadores en una misma instrucción.

Para observar cómo Python interpreta las operaciones aritméticas, el siguiente código presenta cálculos básicos y un ejemplo de precedencia. Este bloque permite verificar la diferencia entre división real, división entera, módulo y potencia.

Código. Operadores aritméticos y precedencia en Python

```
print(10 + 3)    # 13

print(10 - 3)    # 7

print(10 * 3)    # 30

print(10 / 3)    # 3.3333333333333335

print(10 // 3)   # 3

print(10 % 3)    # 1

print(2 ** 5)    # 32

# Precedencia

print(2 + 3 * 4)  # 14

print((2 + 3) * 4) # 20
```

Resultado esperado

13

7

30

3.3333333333333335

3

1

32

14

20

Este ejemplo muestra que Python respeta el orden matemático de las operaciones: primero calcula potencias, luego multiplicaciones, divisiones y módulos, y finalmente sumas o restas. Al usar paréntesis, la operación encerrada se resuelve primero, lo que permite controlar el resultado de una expresión.

Los operadores de asignación compuesta permiten actualizar el valor de una variable mediante una operación abreviada. En lugar de escribir una expresión completa, como `x = x + 5`, Python permite usar `x += 5`. Esta forma reduce la extensión del código y facilita la lectura cuando una variable debe modificarse durante la ejecución del programa.

Código. Operadores de asignación compuesta en Python

```
x = 10

x += 5    # x = x + 5

print(x)

x -= 3    # x = x - 3

print(x)

x *= 2    # x = x * 2

print(x)

x /= 4    # x = x / 4
```

```
print(x)

x //= 2    # x = x // 2

print(x)

x **= 2    # x = x ** 2

print(x)
```

Resultado esperado

```
15
12
24
6.0
3.0
9.0
```

Este ejemplo permite observar cómo una misma variable cambia de valor a medida que se aplican operadores de asignación compuesta. Esta forma abreviada facilita la escritura de instrucciones cuando se requiere actualizar contadores, acumuladores, totales o valores intermedios dentro de un programa.

La siguiente tabla organiza los operadores aritméticos más utilizados. Para mejorar la lectura, el ejemplo de uso y el resultado esperado se presentan en columnas separadas.

Tabla 8. Operadores aritméticos y precedencia en Python

Operador	Nombre	Precedencia general	Ejemplo de uso	Resultado esperado
**	Potencia	Mayor	2 ** 3	8
*	Multiplicación	Alta	5 * 3	15
/	División real	Alta	7 / 2	3.5
//	División entera	Alta	7 // 2	3
%	Módulo o residuo	Alta	7 % 2	1
+	Suma	Media	5 + 3	8
-	Resta	Media	5 - 3	2

El dominio de estos operadores permite construir expresiones matemáticas claras, precisas y coherentes con el cálculo esperado. Cuando una operación combina varios operadores, se recomienda usar paréntesis para facilitar la lectura y evitar ambigüedades.

Una vez comprendido cómo Python realiza cálculos numéricos, es necesario estudiar los operadores que permiten comparar valores y evaluar condiciones. Estos operadores son la base para construir expresiones que devuelven resultados lógicos como True o False.

5.2. Operadores lógicos, relacionales y de cadena

Los operadores lógicos, relacionales y de cadena permiten comparar valores, combinar condiciones y trabajar con textos dentro de un programa en Python. Su uso

es fundamental para construir expresiones que devuelven resultados booleanos, validar información ingresada por el usuario y preparar reglas que orientan la ejecución del programa.

En la práctica, estos operadores suelen utilizarse de forma conjunta. Los operadores relacionales comparan dos valores; los operadores lógicos combinan una o varias condiciones, y los operadores de cadena permiten unir, repetir o verificar la presencia de un texto dentro de otro. Esta integración facilita la construcción de instrucciones más claras y funcionales.

Para comprender su aplicación, se presentan primero los operadores lógicos más utilizados en Python. Estos operadores permiten evaluar condiciones compuestas y determinar si una expresión debe considerarse verdadera o falsa.

Tabla 9. Operadores lógicos en Python

Operador	Descripción	Ejemplo de uso	Resultado esperado
and	Devuelve True si ambas condiciones son verdaderas.	$(5 > 3)$ and $(2 < 4)$	True
or	Devuelve True si al menos una condición es verdadera.	$(5 > 3)$ or $(2 > 4)$	True
not	Invierta el valor lógico de una condición.	not $(5 > 3)$	False

Esta información permite interpretar condiciones compuestas y obtener el resultado lógico de una expresión. El uso adecuado de and, or y not favorece la construcción de reglas claras, especialmente cuando un programa debe evaluar más de un criterio.

Los operadores relacionales, también llamados operadores de comparación, permiten establecer relaciones entre dos valores. Su resultado siempre es booleano: True, cuando la comparación se cumple, o False, cuando no se cumple. Es importante diferenciar el operador de asignación = del operador de igualdad ==, ya que el primero guarda un valor en una variable y el segundo compara dos valores.

Para diferenciar estos operadores, la siguiente tabla presenta el símbolo, su nombre, un ejemplo de uso y el resultado esperado.

Tabla 10. Operadores relacionales en Python

Operador	Nombre	Ejemplo de uso	Resultado esperado
==	Igual a	5 == 5	True
!=	Diferente de	5 != 3	True
>	Mayor que	5 > 3	True
<	Menor que	5 < 3	False
>=	Mayor o igual que	5 >= 5	True
<=	Menor o igual que	3 <= 5	True

Los ejemplos presentados en la tabla anterior permiten reconocer cómo Python compara dos valores y devuelve un resultado booleano. Esta distinción es importante porque evita confundir el operador de asignación = con el operador de igualdad ==: el primero guarda un valor en una variable, mientras que el segundo verifica si dos valores son iguales.

A partir de estas comparaciones, los operadores relacionales pueden combinarse con operadores lógicos para evaluar varias condiciones en una misma expresión. El siguiente código presenta un caso sencillo en el que se revisan dos datos: la edad y el salario. Con estos valores, Python determina si las condiciones se cumplen y muestra el resultado en la consola.

Código. Ejemplo combinado de operadores lógicos y relacionales

```
# Ejemplo combinado de operadores lógicos y relacionales
```

```
edad = 25
```

```
salario = 2500000
```

```
print((edad >= 18) and (salario > 1000000))
```

```
print((edad < 18) or (salario > 1000000))
```

```
print(not (edad < 18))
```

Resultado esperado

```
True
```

```
True
```

```
True
```

El resultado evidencia que las expresiones evaluadas son verdaderas. En la primera línea, se cumplen las dos condiciones: la edad es mayor o igual a 18 y el salario supera 1.000.000. En la segunda línea, basta con que una condición sea verdadera para

que or devuelva True. En la tercera línea, not invierte la condición `edad < 18`, que era falsa, y por eso el resultado final es True.

Además de comparar números y combinar condiciones, Python permite trabajar con cadenas de caracteres. Estos operadores facilitan unir textos, repetir mensajes y verificar si una palabra o fragmento aparece dentro de una cadena. Esta funcionalidad es útil cuando se procesan nombres, códigos, respuestas del usuario o mensajes generados por el programa.

A. Operadores de cadena

Además de comparar números y combinar condiciones, Python permite trabajar con cadenas de caracteres. Estos operadores facilitan unir textos, repetir mensajes y verificar si una palabra o fragmento aparece dentro de otra cadena. Esta funcionalidad es útil cuando se procesan nombres, códigos, respuestas del usuario o mensajes generados por el programa.

Para comprender su uso, se presentan las operaciones más frecuentes con cadenas de caracteres:

- **Concatenación con +:** permite unir dos o más cadenas en una sola salida. Por ejemplo, `"Hola " + "Mundo"` genera el mensaje `Hola Mundo`.
- **Repetición con *:** permite repetir una cadena el número de veces indicado. Por ejemplo, `"Ja" * 3` genera `JaJaJa`.
- **Pertenencia con in:** verifica si una palabra, letra o fragmento aparece dentro de otra cadena. Por ejemplo, `"Python" in "Aprende Python hoy"` devuelve True.

- **No pertenencia con not in:** verifica si una palabra, letra o fragmento no aparece dentro de otra cadena. Por ejemplo, "Java" not in "Aprende Python hoy" devuelve True.

Estas operaciones permiten manipular texto de forma sencilla y son útiles para construir mensajes, validar respuestas o revisar información ingresada por el usuario.

Código. Operadores de concatenación, repetición y pertenencia

Concatenación de cadenas

```
print("Hola " + "Mundo")
```

Repetición de cadenas

```
print("Ja" * 3)
```

```
print("-" * 40)
```

Pertenencia en cadenas

```
print("Python" in "Aprende Python hoy")
```

```
print("Java" not in "Aprende Python hoy")
```

Resultado esperado

Hola Mundo

JaJaJa

True

True

El resultado muestra que Python puede unir cadenas, repetirlas y verificar si un texto está presente o ausente dentro de otro. Estas operaciones fortalecen el manejo de información textual y facilitan la construcción de salidas más claras en los programas.

Después de reconocer cómo los operadores permiten calcular, comparar y manipular textos, se estudian las funciones integradas de Python, útiles para resolver tareas frecuentes como recibir datos, mostrar resultados, convertir valores y trabajar con secuencias de forma clara y eficiente.

5.3. Funciones integradas para entrada, salida y secuencias

Python dispone de funciones integradas, conocidas en inglés como built-in functions, que están disponibles sin importar módulos adicionales. Estas funciones forman parte del núcleo del lenguaje y pueden utilizarse directamente en cualquier programa para realizar tareas frecuentes, como calcular valores, convertir tipos de datos, consultar información de objetos, recibir entradas, mostrar salidas y manipular secuencias.

El uso de estas funciones permite escribir código más claro, breve y eficiente. En lugar de crear instrucciones desde cero para resolver tareas comunes, el programador puede utilizar funciones ya disponibles en Python, lo que facilita la lectura, la depuración y el mantenimiento del programa.

Para organizar su estudio, las funciones integradas se pueden agrupar según el propósito que cumplen dentro de un programa:

a. **Funciones matemáticas básicas:** permiten realizar operaciones matemáticas comunes sin necesidad de importar el módulo `math`. Entre las más utilizadas se encuentran:

- **`abs()`:** devuelve el valor absoluto de un número.
- **`max()`:** retorna el valor máximo de una secuencia o conjunto de argumentos.
- **`min()`:** retorna el valor mínimo de una secuencia o conjunto de argumentos.
- **`sum()`:** calcula la suma total de los elementos de una secuencia numérica.
- **`round()`:** redondea un número al entero más cercano o al número de decimales indicado.
- **`pow()`:** calcula la potencia de un número; equivale al operador `**`.

b. **Funciones de información:** se emplean para obtener datos sobre los objetos durante la ejecución del programa. Son útiles al momento de depurar el código, validar tipos de datos y comprender cómo Python interpreta cada valor:

- **`type()`:** devuelve el tipo de un objeto.
- **`isinstance()`:** verifica si un objeto pertenece a un tipo determinado.
- **`id()`:** retorna la dirección de memoria donde se encuentra el objeto.
- **`len()`:** devuelve la longitud o cantidad de elementos de una secuencia.
- **`dir()`:** lista todos los atributos y métodos disponibles para un objeto.

- **help()**: muestra la documentación de un objeto, función o módulo.

c. **Funciones de conversión de tipos:** permiten convertir un valor de un tipo de dato a otro. Son indispensables al trabajar con la función `input()`, ya que el valor ingresado por el usuario siempre llega como cadena de caracteres:

- **int()**: convierte un valor a entero.
- **float()**: convierte un valor a número de punto flotante.
- **str()**: convierte un valor a cadena de caracteres.
- **bool()**: convierte un valor a su equivalente booleano (True o False).
- **list()**: convierte una secuencia en una lista.
- **tuple()**: convierte una secuencia en una tupla.
- **set()**: convierte una secuencia en un conjunto y elimina duplicados.

Para facilitar la consulta de las funciones integradas más utilizadas, la siguiente tabla las organiza según su propósito dentro de un programa. Esta clasificación permite identificar qué función usar cuando se requiere calcular, consultar información, convertir datos, interactuar con el usuario o manipular secuencias

Para ampliar el reconocimiento de las funciones integradas de Python, se recomienda consultar la documentación oficial del lenguaje.

Enlace de consulta. <https://docs.python.org/3/library/functions.html>

Aunque las funciones integradas permiten resolver muchas tareas habituales, algunos problemas requieren funcionalidades más especializadas. Para estos casos, Python organiza el código en módulos y librerías reutilizables, los cuales amplían las capacidades del lenguaje y permiten desarrollar programas más completos.

6. Módulos, librerías y estándares de codificación

Los módulos, las librerías y los estándares de codificación permiten escribir programas en Python de forma más organizada, reutilizable y fácil de mantener. Estos recursos ayudan a resolver tareas comunes, evitar la repetición de instrucciones y presentar el código con una estructura comprensible para otras personas.

Para comprender su función dentro de un programa, es importante reconocer cómo cada elemento aporta a la construcción de soluciones más claras y eficientes:

- **Módulos:** son archivos de Python que reúnen funciones, clases o variables para reutilizarlas en otros programas. Por ejemplo, en una empresa que calcula descuentos de ventas, un módulo puede guardar las funciones de cálculo para usarlas en diferentes reportes sin escribirlas de nuevo.
- **Librerías:** son conjuntos organizados de módulos que permiten resolver tareas más amplias o especializadas. Por ejemplo, una empresa que analiza ventas mensuales puede usar una librería para organizar datos, generar cálculos y presentar resultados sin construir todas las funciones desde cero.
- **Importaciones:** permiten incorporar módulos o funciones dentro de un programa mediante la sentencia `import`. Por ejemplo, si se requiere calcular el área de un círculo, se puede importar un módulo matemático para usar el valor de π y evitar escribirlo manualmente.
- **Estándares de codificación:** son recomendaciones para escribir código claro, ordenado y fácil de revisar. Por ejemplo, usar nombres como

`total_ventas` o `nombre_cliente` ayuda a comprender rápidamente qué información almacena cada variable dentro del programa.

- **PEP 8:** es la guía de estilo oficial de Python para mejorar la legibilidad del código. Por ejemplo, recomienda organizar las importaciones al inicio del archivo, usar cuatro espacios por nivel de indentación y nombrar las variables con palabras descriptivas.

Estos elementos permiten construir programas más comprensibles y sostenibles. Al usar módulos, librerías y estándares de codificación, se fortalecen buenas prácticas de programación y se facilita que otras personas revisen, mantengan o amplíen el código.

La programación moderna no consiste en escribir cada línea de código desde cero. Python se basa en la filosofía "Batteries Included" (Baterías incluidas), lo que significa que el lenguaje ya viene equipado con herramientas listas para usar.

Esta estructura permite a los desarrolladores mantener un código limpio, reutilizable y fácil de mantener, dividiendo problemas complejos en piezas más pequeñas y manejables. Gracias a este enfoque, el programador puede aprovechar soluciones ya probadas en lugar de reescribirlas, lo que ahorra tiempo y reduce la posibilidad de errores.

6.1. Concepto de módulo, librería y formas de importación

Un módulo en Python es un archivo con extensión `.py` que contiene definiciones de funciones, clases y variables que pueden ser reutilizadas en otros programas. Una

librería (o biblioteca) es un conjunto organizado de módulos que proveen funcionalidades especializadas.

Python incluye una amplia biblioteca estándar (standard library) con cientos de módulos para tareas comunes. La existencia de esta biblioteca tan completa es una de las principales fortalezas del lenguaje, resumida en la expresión "batteries included" (baterías incluidas).

Además de la biblioteca estándar, existe una enorme cantidad de módulos desarrollados por la comunidad que pueden instalarse y reutilizarse, lo que amplía casi sin límites las posibilidades del lenguaje.

A. Tipos de módulos y formas de importación.

No todos los módulos tienen el mismo origen. Según su procedencia, pueden clasificarse en módulos integrados en el propio lenguaje, módulos desarrollados por terceros e instalables por el programador, y módulos propios creados por el desarrollador para su proyecto.

Independientemente de su tipo, para emplear un módulo es necesario importarlo. La forma de hacerlo influye en cómo se invocan posteriormente sus funciones, por lo que conviene conocer las distintas variantes de la sentencia import.

En los siguientes apartados se describen estos tres tipos de módulos y, a continuación, las principales formas de importación con ejemplos prácticos.

En Python se distinguen tres tipos principales de módulos:

- **Módulos integrados (built-in):** forman parte del núcleo de Python.

Ejemplos: math, os, sys, datetime, random.

- **Módulos de terceros:** desarrollados por la comunidad, instalables con pip.
Ejemplos: numpy, pandas, matplotlib, requests.
- **Módulos propios:** archivos .py creados por el programador.

Reconocer estos tres orígenes permite decidir con qué módulo trabajar: usar primero los integrados, recurrir a los de terceros cuando la tarea lo exija y crear módulos propios para organizar el código del proyecto.

A continuación, se ilustra en la tabla el resumen de los tipos de módulos más utilizados en Python:

Para reconocer la utilidad de la biblioteca estándar, la tabla relaciona módulos frecuentes con la funcionalidad principal que ofrecen.

Tabla 11. Módulos integrados de uso frecuente en Python

Módulo	Funcionalidad principal
math	Funciones matemáticas: sqrt, pi, sin, cos, log, factorial.
random	Números aleatorios: random(), randint(), choice(), shuffle().
datetime	Manejo de fechas y horas: date, time, datetime, timedelta.
os	Interacción con el sistema operativo: rutas, directorios.
sys	Información del sistema y del intérprete de Python.
json	Lectura y escritura de archivos JSON.
csv	Lectura y escritura de archivos CSV.
string	Constantes y funciones para manipulación de cadenas.

Estos módulos forman parte de la biblioteca estándar de Python, por lo que están disponibles sin instalaciones adicionales. Conocer su existencia evita "reinventar la rueda" y orienta al programador hacia la herramienta adecuada para cada tarea.

Para utilizar un módulo se emplea la sentencia `import`. Esta sentencia admite distintas formas, según se necesite importar el módulo completo, solo algunas de sus funciones o asignarle un alias más corto:

- **Importación completa:** se importa todo el módulo y sus funciones se invocan anteponiendo el nombre del módulo seguido de un punto. Es la forma más explícita, porque deja claro de qué módulo proviene cada función.
- **Importación de funciones específicas:** se importan únicamente las funciones que se van a usar, las cuales se invocan directamente por su nombre, sin anteponer el del módulo.
- **Importación con alias:** se asigna al módulo un nombre más corto mediante la palabra clave `as`, que se utiliza luego en lugar del nombre original. Es una práctica habitual con librerías de nombre extenso o de uso muy frecuente.

Elegir entre estas tres formas depende del alcance del programa, la importación completa favorece la claridad, la importación de funciones específicas reduce la escritura, y el alias resulta útil con módulos de nombre extenso o de uso frecuente.

Además de saber cómo importar correctamente un módulo, es necesario escribir el código siguiendo un estilo uniforme. El siguiente apartado presenta los estándares de codificación PEP 8, guía oficial para lograrlo.

6.2. Estándares de codificación PEP 8

El PEP 8 es la guía de estilo oficial para la escritura de código Python. Su propósito es establecer convenciones que mejoren la legibilidad del código y faciliten la colaboración entre programadores.

- **Indentación:** se deben utilizar 4 espacios por nivel de indentación. No se recomienda mezclar tabulaciones con espacios. La indentación define la estructura lógica del programa; un bloque mal indentado producirá un `IndentationError`.
- **Longitud de línea:** las líneas no deben superar los 79 caracteres. Si una expresión es demasiado larga, se puede dividir usando paréntesis o la barra invertida.
- **Nomenclatura:** variables y funciones en `snake_case` (`total_ventas`). Constantes en MAYÚSCULAS (`MAX_INTENTOS`, `IVA`). Clases en `CamelCase` (`MiClase`). Nombres descriptivos, sin abreviaciones ambiguas.
- **Espacios en blanco:** un espacio alrededor de operadores de asignación (`x = 5`) y comparación (`x == 5`). Sin espacios dentro de paréntesis. Dos líneas en blanco antes de funciones de nivel superior.
- **Importaciones:** al inicio del archivo, en tres grupos separados: módulos estándar, módulos de terceros y módulos propios. Se prefiere `import math` sobre `from math import *`.

Estas convenciones fortalecen la legibilidad del código y facilitan que otras personas comprendan, revisen y mantengan los programas escritos en Python.

Para fortalecer la escritura de código claro, ordenado y fácil de mantener, se recomienda consultar la guía oficial del estándar PEP 8. Este recurso orienta buenas

prácticas sobre nombres de variables, uso de espacios, organización de importaciones, comentarios y estructura general del código en Python. <https://peps.python.org/pep-0008/>

Seguir el estándar PEP 8 no responde a una exigencia estética, sino a una práctica que facilita el trabajo colaborativo y la sostenibilidad de un proyecto a lo largo del tiempo. Cuando distintos programadores escriben código bajo las mismas convenciones, cualquier persona del equipo puede leerlo, revisarlo o modificarlo sin necesidad de familiarizarse primero con un estilo particular.

Aplicar estas convenciones desde el inicio del aprendizaje aporta beneficios que van más allá de un solo programa:

- **Facilita la colaboración:** cuando varias personas trabajan sobre un mismo proyecto, seguir las mismas convenciones evita que cada programador nombre las variables o estructure el código a su manera. Por ejemplo, si todo el equipo aplica `snake_case`, cualquiera puede leer una variable como `total_ventas` y entender su propósito, aunque no haya participado en escribirla.
- **Reduce errores:** un código con indentación uniforme y espacios consistentes resulta más fácil de revisar antes de ejecutarlo. Por ejemplo, mantener siempre 4 espacios por nivel de indentación permite detectar a simple vista si una línea quedó fuera del bloque de un `if` o un `for`, antes de que Python genere un error.
- **Prepara para el entorno profesional:** muchas empresas de desarrollo de software exigen que el código cumpla con PEP 8 antes de aceptar una contribución. Por ejemplo, herramientas como los linters revisan

automáticamente el estilo del código durante la revisión de un proyecto y señalan las líneas que no cumplen el estándar.

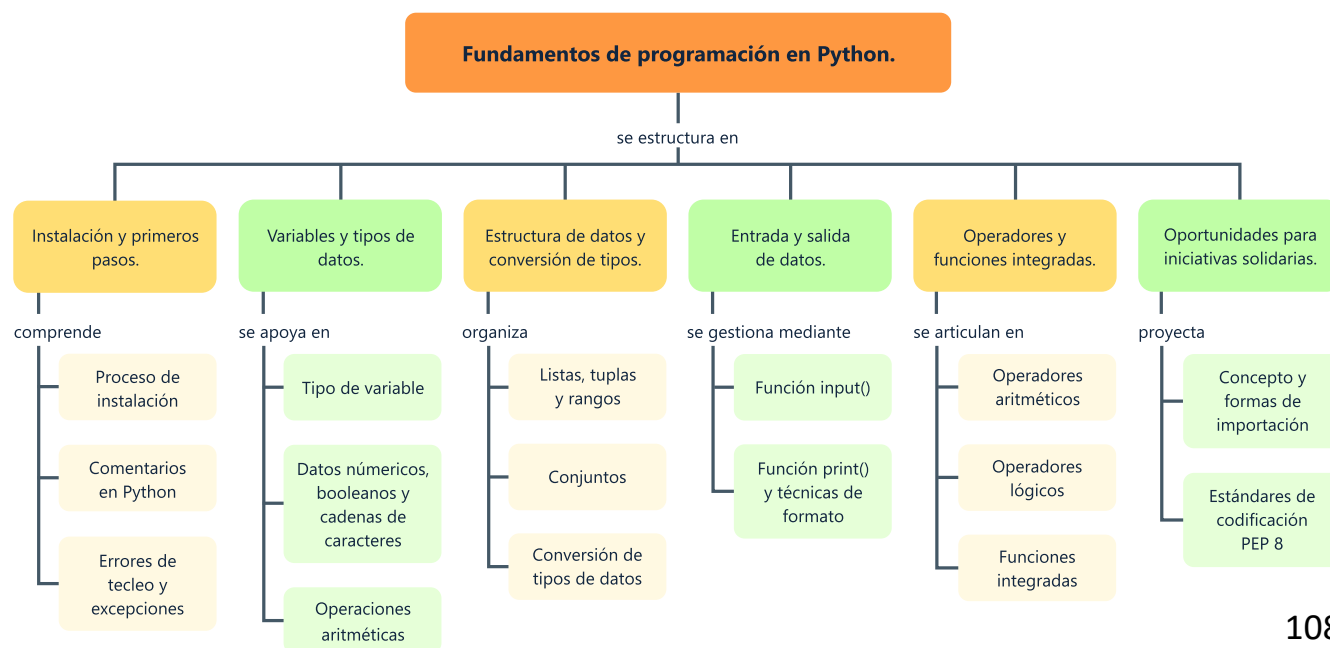
Con la aplicación del estándar PEP 8 se cierra el recorrido por los fundamentos de la programación en Python abordados en este componente formativo. Desde la instalación del entorno de trabajo hasta la escritura de código conforme a un estilo profesional, cada tema recorrido aportó un elemento necesario para construir programas secuenciales claros, funcionales y fáciles de mantener.

El aprendiz cuenta ahora con las bases necesarias para declarar variables, gestionar tipos de datos, capturar y presentar información, aplicar operadores y organizar el código mediante módulos y buenas prácticas de codificación. Estos fundamentos constituyen el punto de partida para avanzar, en los siguientes componentes del programa, hacia el uso de estructuras de control y la construcción de soluciones cada vez más completas.

Síntesis

El componente formativo presenta los fundamentos iniciales del lenguaje Python, con énfasis en la instalación del entorno de trabajo, el uso de comentarios, el reconocimiento de errores, la declaración de variables y el manejo de tipos de datos. A partir de estos elementos, se abordan los datos numéricos, booleanos y cadenas de caracteres, las operaciones aritméticas y las estructuras de datos compuestas, como listas, tuplas, rangos, diccionarios y conjuntos, las cuales permiten almacenar, organizar y transformar información dentro de un programa.

El recorrido continúa con la conversión de tipos de datos, las funciones de entrada y salida, los operadores aritméticos, lógicos, relacionales y de cadena, así como las funciones integradas, los módulos, las librerías y los estándares de codificación PEP 8. De este modo, se construye una base conceptual y práctica para elaborar programas secuenciales claros, funcionales y fáciles de mantener. A continuación, el mapa conceptual organiza de manera visual los ejes temáticos del componente formativo y las relaciones que los articulan.



Glosario

Casting: conversión explícita de un dato a otro tipo mediante funciones como `int()`, `float()` o `str()`.

Concatenación: unión de dos o más cadenas de caracteres en una sola salida mediante el operador `+`.

Función: bloque reutilizable de instrucciones que realiza una tarea específica dentro de un programa.

Indentación: espacio al inicio de una línea que permite organizar bloques de código en Python.

Intérprete: programa que ejecuta las instrucciones de Python línea por línea.

Librería: conjunto organizado de módulos que ofrece funcionalidades relacionadas para resolver tareas específicas.

Módulo: archivo de Python con extensión `.py` que contiene funciones, clases o variables reutilizables.

Operador: símbolo o palabra que permite realizar operaciones sobre uno o más valores.

PEP 8: guía de estilo oficial de Python que establece convenciones de codificación para mejorar la legibilidad del código fuente.

Prompt: mensaje que se muestra al usuario para solicitar la entrada de un dato.

Referencias bibliográficas

Cuevas, A. (2017). Python 3: Curso práctico. Ediciones de la U.

Fazt Tech. (2019). Curso Python para principiantes [Video]. YouTube.

<https://www.youtube.com/watch?v=chPhIsHoEPo>

Guzdial, B., & Vidal, A. (2013). Introducción a la computación y programación con Python. Pearson Educación.

Hinojosa, A. (2016). Python paso a paso. Ediciones de la U.

Lutz, M. (2013). Learning Python (5.^a ed.). O'Reilly Media.

Pérez, A. (2016). Python fácil. Alfaomega Grupo Editor.

Python Software Foundation. (s.f.). The Python tutorial.

<https://docs.python.org/3/tutorial/index.html>

Salazar, P. (2019). Empezando a programar en Python. Editorial Escuela Colombiana de Ingeniería.

Van Rossum, G., Warsaw, B., & Coghlan, N. (2001). PEP 8 – Style guide for Python code.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Solanlly Sánchez Melo	Experta temática	Centro de Comercio y Servicios - Regional Tolima
Gloria Lida Alzate Suárez	Evaluadora instruccional	Centro de Comercio y Servicios - Regional Tolima
José Jaime Luis Tang Pinzón	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Manuel Felipe Echavarria Orozco	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
Jorge Eduardo Rueda Peña	Evaluador de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Javier Mauricio Oviedo	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima